



INFORME DE PROYECTO

“RECICLAJE EFICIENTE” – SISTEMA DE RECICLAJE INTELIGENTE

PRESENTADO POR

ANGIE NATALIA MARTÍNEZ VÁSQUEZ

WILSON NORVEY RODRIGUEZ

UNIVERSIDAD SERGIO ARBOLEDA

ESCUELA DE CIENCIAS EXACTAS E INGENIERIA

INGENIERIA ELECTRÓNICA

DISEÑO DE PRODUCTO 2

BOGOTÁ D.C.

2025

Resumen:

Este informe presenta el proceso de definición, diseño, selección y desarrollo de un proyecto tecnológico orientado a la sostenibilidad. A partir de un análisis inicial de ideas, encuestas de mercado y evaluación de requerimientos, se seleccionó la propuesta **Reciclajeficiente**, un sistema de reciclaje inteligente capaz de clasificar residuos y fomentar la cultura ambiental.

El documento recopila el análisis de mercado, la justificación del proyecto, los requerimientos funcionales y no funcionales, el análisis DOFA, el cronograma de actividades, y los avances en diseño electrónico y de arquitectura del sistema. Asimismo, se incluye un espacio reservado para la representación gráfica de la **arquitectura del proyecto**, tanto física como funcional.

Objetivos:**Objetivo General**

Desarrollar un sistema de reciclaje inteligente, denominado *Reciclajeficiente*, que permita la clasificación automatizada de residuos, promoviendo prácticas de sostenibilidad y contribuyendo a la economía circular.

Objetivos Específicos

1. Analizar diferentes ideas de producto mediante encuestas y seleccionar la más viable.
2. Definir los requerimientos funcionales y no funcionales del sistema de reciclaje inteligente.
3. Elaborar un análisis DOFA para determinar la viabilidad técnica, social y económica del proyecto.
4. Diseñar la arquitectura física y electrónica del sistema, incluyendo la selección de componentes y el diseño de PCB.
5. Implementar prototipos de control y pruebas de circuitos necesarios para el funcionamiento del sistema.
6. Desarrollar un cronograma de actividades para la construcción, ensamble y validación del prototipo.

Introducción:

En este informe se documenta el desarrollo del proyecto “Reciclajeficiente”. El proceso inició con una lluvia de ideas, seguida de un estudio de mercado mediante encuestas a 50 participantes, lo que permitió determinar el nivel de aceptación de diversas propuestas. Posteriormente, se establecieron los criterios de selección y se definieron los requerimientos del sistema.

El documento también presenta el análisis DOFA, la planeación estratégica de actividades y los avances en diseño electrónico, circuitos de prueba y PCB.

Desarrollo del Proyecto:

1. Definiciones y Conceptos Iniciales

- Concepto de proyecto [1].
- Stakeholders y requerimientos.
- Importancia del alcance del proyecto.

2. Generación y Selección de Ideas

- Listado de ideas iniciales.
- Resultados de encuestas y estadísticas de aceptación.
- Selección de *Reciclajeficiente* como propuesta principal.

3. Requerimientos

- Requerimientos funcionales.
- Requerimientos no funcionales.

4. Análisis DOFA

- Fortalezas, oportunidades, debilidades y amenazas.
- Conclusión: *Reciclajeficiente* es más viable que alternativas como *SmartEyes*.

5. Planeación

- Cronograma de actividades en Planner.
- Etapas desde planteamiento del problema hasta entrega final.

6. Diseño de Arquitectura

- Diagrama de arquitectura.

- Bocetos y diseño en 3D.
- Listado de componentes y proveedores.
- Diagramas de bloques de funcionamiento.

7. Desarrollo de Circuitos y Pruebas

- Actividades en clase (control de encendido/apagado con MOSFET).
- Implementación de switcheo entre fuente y batería.
- Diseño de PCB y recomendaciones técnicas.

8. Programación

- Backend, con comunicación entre servidores, información y desarrollo de base de datos.
- Frontend, desarrollo de página web con el framework VUE.JS.
- Lógica de programación de funcionamiento principal del circuito.

Contenido:

Capítulo 1. Conceptos Fundamentales del Proyecto	7
• Definición de proyecto • Stakeholders • Requerimientos funcionales y no funcionales • Alcance del proyecto	
Capítulo 2. Generación y Selección de Ideas	10
• Lluvia de ideas iniciales • Encuestas y análisis de resultados • Selección de propuestas finalistas	
Capítulo 3. Requerimientos de los Proyectos Seleccionados	43
• Requerimientos del proyecto <i>Reciclajeficiente</i> • Requerimientos del proyecto <i>SmartEyes</i> • Comparativa y análisis	
Capítulo 4. Análisis DOFA	46
• DOFA de <i>Reciclajeficiente</i> • DOFA de <i>SmartEyes</i> • Conclusiones de viabilidad	
Capítulo 5. Diseño de Arquitectura	50
• Bocetos iniciales • Diseño en 3D • Listado de componentes y proveedores • Diagrama de arquitectura • Diagrama de bloques de funcionamiento	
Capítulo 6. Actividad en Clase 1	54
• Implementación inicial en ESP32 • Prueba de control de encendido/apagado	

Capítulo 7. Actividad en Clase 2	57
• Implementación de apagado con pulsador (10 segundos) • Esquemático y explicación del funcionamiento	
Capítulo 8. Actividad en Clase 3	60
• Implementación de encendido/apagado con pulsador (2 segundos) • Circuito y análisis de señales	
Capítulo 9. Gestión de Energía y Baterías	61
• Uso de BMS • Switcheo automático entre fuente externa y batería • Recomendaciones para la protección del sistema	
Capítulo 10. Diseño de PCB	63
• Recomendaciones técnicas • Diseño preliminar • Circuito completo con alimentación y pulsador • Power Path Management • Esquemático Altium • PCB Altium	
Capítulo 11. Programación Backend – Frontend	75
• Desarrollo Backend (actividad en clase 4, 5 y 6) • Uso de dbdiagram.io, Postman y MongoDB Atlas • Desarrollo Frontend • Uso de VUE.JS • Programación en C lógica de funcionamiento del sistema	

Capítulo 1

Se visualizó el proceso de definir, diseñar, y producir un producto final, además de tener en cuenta definiciones como que es un proyecto, un stakeholder, un alcance del proyecto, los requerimientos, como es un proceso de negocio, etc.

¿Qué es un proyecto?

Según el Project Management Institute (PMI), un proyecto es un esfuerzo temporal emprendido para crear un entregable singular, el cual, se basa de un proceso único consistente en un conjunto de actividades coordinadas y controladas con fechas de inicio y fin, llevadas a cabo para lograr un objetivo. Existen tres tipos de proyecto:

1. **Temporal:** Limitado en el tiempo.
2. **Entregable:** Bien o servicio (Entregables definidos y acordados, o un Producto).
3. **Único:** No se ha hecho antes.

Quienes están involucrados activamente en el proyecto se denominan **stakeholders**, sin embargo, pueden generar positiva o negativamente en el desarrollo del proyecto y sus resultados.

Para saber las necesidades, deseos y expectativas que se tienen en un proyecto, se requiere recopilar los **requerimientos**, estos mismos nos permiten saber si un proyecto es viable o no. También nos ayudan a definir y manejar las expectativas del cliente. Estos incluyen: necesidades y expectativas, cuantificadas y documentadas de los stakeholder.

Estos se dividen en dos tipos:

- **Requerimientos funcionales:** Relacionados con el comportamiento del Producto del Proyecto (Funcionalidad, reglas, interfaces, informes a producir, información histórica).
- **Requerimientos no-funcionales:** Relacionados con los atributos o propiedades del Producto del Proyecto (Desempeño, Cualidades y Restricciones).

El **alcance del proyecto** es una descripción de las características del Producto para cuya creación se emprendió el Proyecto y debe proveer suficiente detalle para soportar la planeación del alcance, su resultado es la **declaración de alcance**.

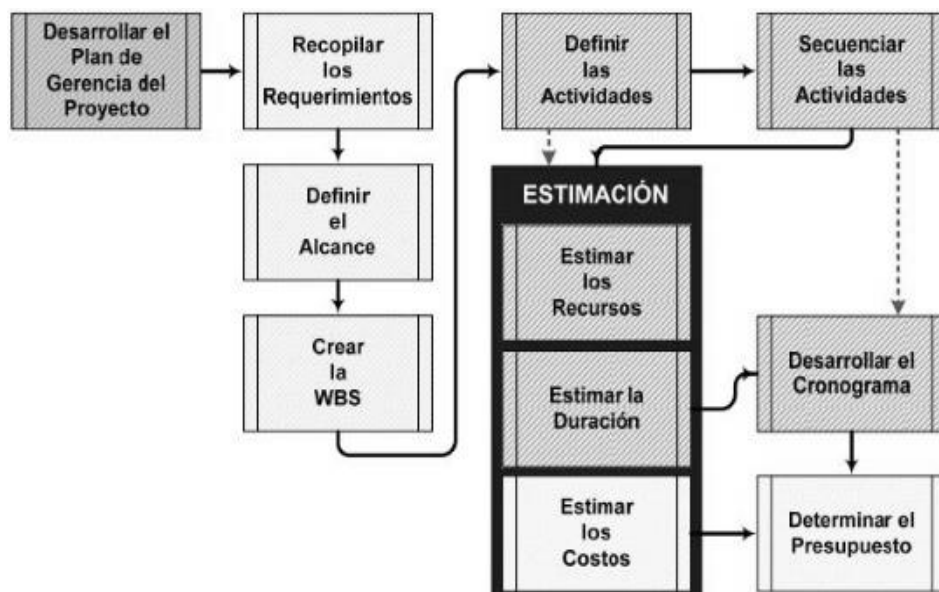
El **desarrollo de un nuevo producto** consiste en un proceso completo que se dedica a crear y llevar un nuevo producto al mercado. Para ello, se deben llevar a cabo dos etapas necesarias:

- Ingeniería de producto.
- Análisis de mercado.

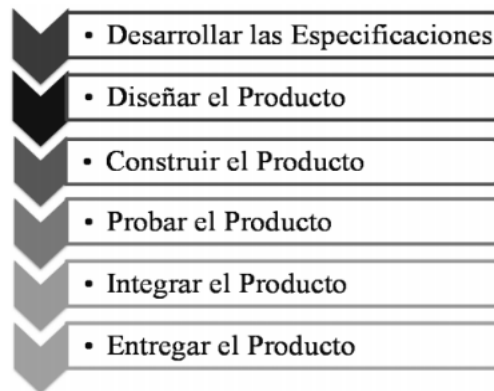
También para que este desarrollo sea eficaz, se requieren de los siguientes pasos:

1. **Generación de productos** (Lluvia de ideas).
2. **Filtrado de la idea eliminación de conceptos que no encajan** (Preguntas clave: ¿El mercado objetivo se beneficiará del producto?, ¿Es técnicamente viable fabricar este producto?, ¿Generará beneficios el producto?).
3. **Desarrollo del concepto y prueba** (Estudio de mercado).
4. **Análisis de Negocios** (Estimar precio de venta, volúmenes de venta y beneficios).
5. **Test de mercado y test Beta** (Prototipo físico y pruebas con posibles ajustes).
6. **Implementación Técnica** (Iniciación del nuevo proyecto).
7. **Comercialización** (Lanzamiento del producto).

Procesos básicos de planeación



Pasos para seguir de un producto en un proyecto



Desarrollo de un proyecto desde el concepto al mercado

From Maker to Market



Capítulo 2

Se realizó una lluvia de ideas para realizar un producto con vistas a resolver un problema, de cada una de ellas se realizó una encuesta para preguntar a las personas de a pie que opinaban del producto, que cambios le harían, si lo comprarían o en qué precio estarían dispuesto a pagar por él.

Las ideas fueron las siguientes:

1. **KeyFinder Pro** (Buscador de objetos personales).
2. **MediCare+** (Dispensador de medicamentos).
3. **CoolAir Auto** (Ventilador automático por temperatura).
4. **SunTrack** (Panel solar con seguidor).
5. **JumpStart 12V** (Batería portátil de arranque).
6. **SmartEyes** (Gafas inteligentes).
7. **GoodClothing** (Closet inteligente con gestión de prendas).
8. **NiceLife** (Sistema de asistencia para personas mayores).
9. **Reciclajeficiente** (Sistema de reciclaje inteligente).
10. **HappyPlants** (Invernadero autónomo con control de clima).

Las preguntas para cada propuesta fueron las siguientes:

KeyFinder Pro (Buscador de objetos personales).

¿Con qué frecuencia pierde objetos como llaves o billeteras?



- ☐ Nunca
- ☐ Rara vez
- ☐ A veces
- ☐ Frecuente
- ☐ Muy frecuente

¿Le interesaría un dispositivo que suene y parpadee para ayudarle a encontrar objetos?

- ☐ Si
- ☐ No

¿Preferiría que el dispositivo funcione sólo con un botón físico o también con una app en el celular?

- ☐ Sólo botón
- ☐ Sólo app
- ☐ Ambos

¿Cuánto estaría dispuesto a pagar por uno? (marcar una opción)

- ☐ Menos de \$20,000
- ☐ \$20,000–\$40,000
- ☐ \$40,000–\$80,000
- ☐ \$80,000–\$150,000
- ☐ Más de \$150,000

¿Le parecería útil que tuviera luz para ubicar en la oscuridad?

- ☐ Si
- ☐ No

¿Dónde compraría este producto si estuviera disponible?

- ☐ Tienda física
- ☐ Tienda online
- ☐ Mercado local
- ☐ No compraría

¿Qué mejora le gustaría ver en este producto? (respuesta abierta) *

Tu respuesta _____

MediCare+ (Dispensador de medicamentos).

¿Alguna persona en su hogar toma medicamentos con regularidad?



- ☐ Sí
- ☐ No

¿Con qué dificultad recuerdan tomar su medicamento a la hora correcta?

- ☐ Nada
- ☐ Poca
- ☐ Moderada
- ☐ Mucha

¿Le interesaría un dispensador que libere la pastilla automáticamente a la hora programada?

- ☐ Sí
- ☐ No

¿Cuánta confianza le daría a un dispositivo para dosificar medicamentos? donde 5 es el mas confiable

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

¿Qué funciones considera indispensables? (marcar hasta 3)

- ☐ Pantalla fácil
- ☐ Bloqueo para niño
- ☐ Notificación al celular
- ☐ Registro de tomas
- ☐ Control por voz

¿Cuánto estaría dispuesto a pagar?

- ☐ Menos de \$80,000
- ☐ \$80,000–\$180,000
- ☐ \$180,000–\$350,000
- ☐ Más de \$350,000

¿Qué cambio sugeriría para que lo comprara para su hogar? (abierta) *

Tu respuesta _____

CoolAir Auto (Ventilador automático por temperatura).

¿En su hogar sufren por altas temperaturas?



- ☐ Sí
- ☐ No

¿Usa ventilador con frecuencia?

- ☐ Sí, diario
- ☐ Algunas veces
- ☐ Raro
- ☐ Nunca

¿Consideraría comprar un ventilador que se enciende solo cuando la temperatura sube?

- ☐ Sí
- ☐ No

¿Qué ahorro mensual estaría dispuesto a aceptar como beneficio (p. ej. menor consumo de energía)

- ☐ Ninguno
- ☐ Poco
- ☐ Moderado
- ☐ Mucho

¿Preferiría un ventilador con opción a energía solar

- ☐ Sí
- ☐ No
- ☐ No se

¿Cuánto pagaría por un ventilador inteligente (un solo equipo)?

- ☐ Menos de \$60,000
- ☐ \$60,000–\$120,000
- ☐ \$120,000–\$240,000
- ☐ Más de \$240,000

¿Qué mejora le haría más útil este ventilador? (abierta) *

Tu respuesta

SunTrack (Panel solar con seguidor).

¿Tiene paneles solares o conoce a alguien que los tenga?



- ☐ Si
- ☐ No

¿Conoce o le interesa la energía solar para reducir gastos de electricidad?

- ☐ Si
- ☐ No

¿Cuál sería su principal motivación para instalarlo? (marcar una)

- ☐ Ahorrar en la factura
- ☐ Independencia energética
- ☐ Sustentabilidad
- ☐ Otro

¿Cuánto estaría dispuesto a pagar adicional por el sistema de seguimiento (como inversión única)?

- ☐ Menos de \$400,000
- ☐ \$400,000–\$1,000,000
- ☐ \$1,000,000–\$3,000,000
- ☐ Más de \$3,000,000

Comentarios o mejoras que usted sugiere (abierta). *

Tu respuesta

JumpStart 12V (Batería portátil de arranque).

¿Ha quedado alguna vez varado por la batería del vehículo



- ☐ Si
- ☐ No

¿Qué tan útil le resultaría un dispositivo que arranca su carro sin ayuda de otro (1-5)

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

¿Preferiría que el dispositivo tenga linterna y puertos USB?

- ☐ Si
- ☐ No

¿Cuánto estaría dispuesto a pagar por este dispositivo?

- ☐ Menos de \$150,000
- ☐ \$150,000–\$300,000
- ☐ \$300,000–\$600,000
- ☐ Más de \$600,000

¿Dónde lo compraría?

- ☐ Tienda autoparte
- ☐ Online
- ☐ Ferretería
- ☐ Otro

Sugerencias o mejoras (abierta). *

Tu respuesta

SmartEyes (Gafas inteligentes).



¿Le interesaría adquirir unas gafas inteligentes como "SmartEyes", que mejoran la *
visión y detectan obstáculos?

- ☐ Sí, definitivamente
- ☐ Sí, tal vez
- ☐ No estoy seguro
- ☐ No me interesa

¿Qué función considera más importante en las "SmartEyes"? *

- ☐ Zoom con gestos manuales para ampliar la imagen
- ☐ Detección de obstáculos con vibración
- ☐ Alarma sonora (buzzer) ante obstáculos peligrosos
- ☐ Todas las anteriores

¿Qué rango de precio estaría dispuesto(a) a pagar por este producto? *

- ☐ Menos de \$100.000 COP
- ☐ Entre \$100.000 y \$200.000 COP
- ☐ Entre \$200.000 y \$300.000 COP
- ☐ Más de \$300.000 COP

¿Preferiría que el aviso de obstáculos sea...? *

- ☐ Solo con vibración
- ☐ Solo con sonido (buzzer)
- ☐ Ambos (vibración + sonido)
- ☐ Depende de la situación (opción para desactivar sonido o vibración)

¿Qué tan importante considera que las gafas tengan diseño liviano y estético? *

- ☐ Muy importante
- ☐ Importante
- ☐ Poco importante
- ☐ No es importante

¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

Tu respuesta

GoodClothing (Closet inteligente con gestión de prendas).

¿Le interesaría adquirir un Closet Inteligente que organice y sugiera combinaciones de ropa automáticamente? *

- ☐ Sí, definitivamente
- ☐ Sí, tal vez
- ☐ No estoy seguro
- ☐ No me interesa

¿Qué característica le parece más útil en un Closet Inteligente? *

- ☐ Identificación de prendas
- ☐ Sugerencias de combinaciones según el clima
- ☐ Registro en app de las prendas disponibles
- ☐ Todas las anteriores

¿Qué rango de precio estaría dispuesto(a) a pagar por este producto? *

- ☐ Menos de \$500.000 COP
- ☐ Entre \$500.000 y \$1.000.000 COP
- ☐ Entre \$1.000.000 y \$1.500.000 COP
- ☐ Más de \$1.500.000 COP

¿Le gustaría que el closet se conecte a una app para mostrar estadísticas de uso de su ropa? *

- ☐ Sí, me parece muy útil
- ☐ Tal vez, dependiendo de las funciones
- ☐ No me interesa

¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

Tu respuesta

NiceLife (Sistema de asistencia para personas mayores).

¿Considera útil un dispositivo portátil que ayude a personas mayores a pedir ayuda y que detecte caídas automáticamente? *

- ☐ Sí, definitivamente
- ☐ Sí, tal vez
- ☐ No estoy seguro
- ☐ No me interesa

¿Qué función considera más importante en este sistema? *

- ☐ Botón de pánico para emergencias
- ☐ Detección automática de caídas
- ☐ Monitoreo de ritmo cardíaco y oxigenación
- ☐ Ubicación GPS para familiares

¿Qué rango de precio estaría dispuesto(a) a pagar por este producto? *

- ☐ Menos de \$200.000 COP
- ☐ Entre \$200.000 y \$400.000 COP
- ☐ Entre \$400.000 y \$600.000 COP
- ☐ Más de \$600.000 COP

¿Cómo preferiría que el dispositivo envíe las alertas? *

- ☐ Mensaje de texto (SMS)
- ☐ Llamada automática a un contacto
- ☐ Notificación en una app móvil
- ☐ Todas las anteriores

¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

Tu respuesta

Reciclajeficiente (Sistema de reciclaje inteligente).

¿Le interesaría un sistema que clasifique automáticamente la basura en plásticos, metales y orgánicos? *

- ☐ Sí, definitivamente
- ☐ Sí, tal vez
- ☐ No estoy seguro

¿Dónde considera más útil instalar este sistema? *

- ☐ Hogares
- ☐ Oficinas
- ☐ Escuelas / universidades
- ☐ Espacios públicos (parques, centros comerciales)

¿Le gustaría que el sistema muestre en una pantalla o app las estadísticas de reciclaje realizadas? *

- ☐ Sí, me parece muy útil
- ☐ Tal vez, dependiendo del diseño
- ☐ No me interesa

¿Estaría dispuesto(a) a usar este sistema de forma frecuente para contribuir al medio ambiente? *

- ☐ Sí, todos los días
- ☐ Sí, pero ocasionalmente
- ☐ No estoy seguro
- ☐ No

¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

Tu respuesta

HappyPlants (Invernadero autónomo con control de clima).

¿Le interesaría un sistema de invernadero autónomo que controle temperatura, humedad, luz y riego automáticamente? *

- ☐ Sí, definitivamente
- ☐ Sí, tal vez
- ☐ No estoy seguro
- ☐ No me interesa

¿Le gustaría que "HappyPlants" guarde un historial y gráficos de crecimiento y condiciones ambientales? *

- ☐ Sí, me parece muy útil
- ☐ Tal vez, dependiendo del costo
- ☐ No me interesa

¿Estaría dispuesto(a) a usar este sistema de forma continua para optimizar el cuidado de sus plantas? *

- ☐ Sí, todos los días
- ☐ Sí, pero ocasionalmente
- ☐ No estoy seguro
- ☐ No

¿Qué rango de precio estaría dispuesto(a) a pagar por este producto? *

- ☐ Menos de \$200.000 COP
- ☐ Entre \$200.000 y \$500.000 COP
- ☐ Entre \$500.000 y \$1.000.000 COP
- ☐ Más de \$1.000.000 COP

¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

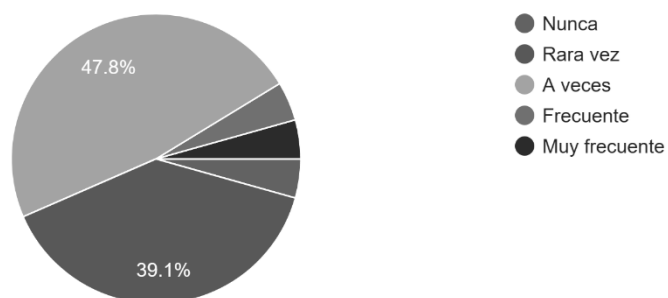
Tu respuesta

Gracias a la herramienta Google Forms, la cual permitió la realización de las anteriores encuestas a 50 personas, también se logró obtener estadísticas visuales acerca de las respuestas obtenidas en cada pregunta. Las cuales fueron las siguientes (las estadísticas a continuación nos ayudarán más adelante en la realización de los requerimientos del producto a realizar).

KeyFinder Pro (Buscador de objetos personales).

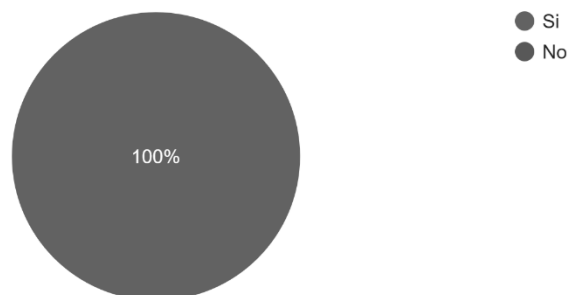
¿Con qué frecuencia pierde objetos como llaves o billeteras?

23 respuestas



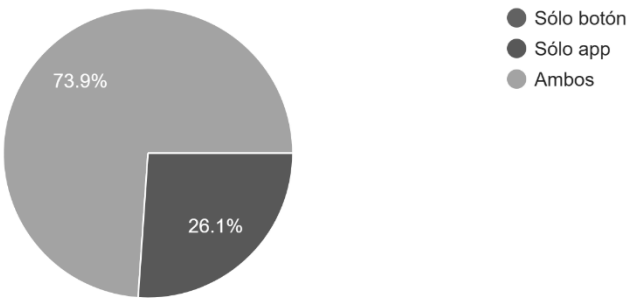
¿Le interesaría un dispositivo que suene y parpadee para ayudarle a encontrar objetos?

23 respuestas



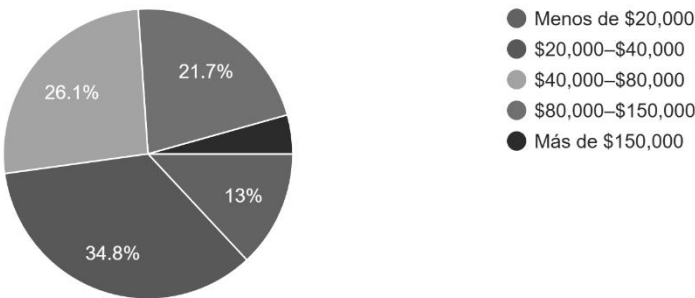
¿Preferiría que el dispositivo funcione sólo con un botón físico o también con una app en el celular?

23 respuestas



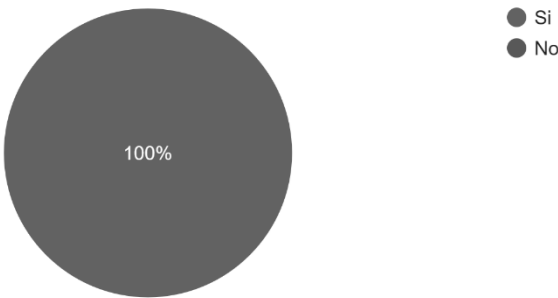
¿Cuánto estaría dispuesto a pagar por uno? (marcar una opción)

23 respuestas



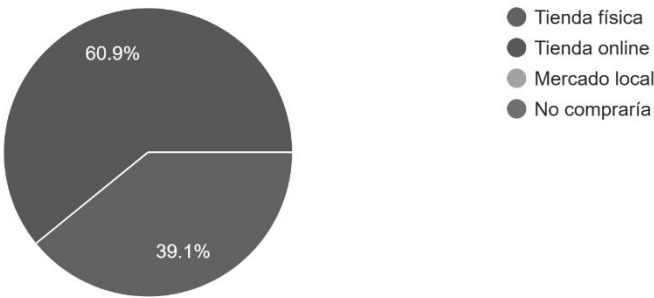
¿Le parecería útil que tuviera luz para ubicar en la oscuridad?

22 respuestas



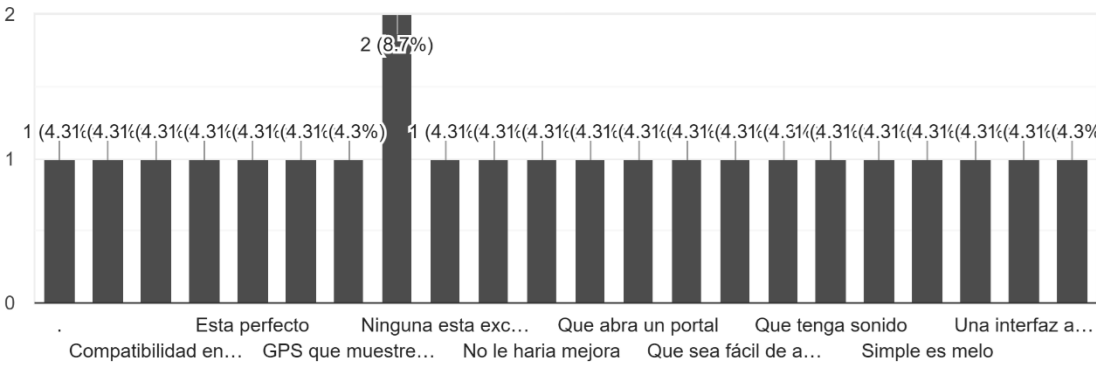
¿Dónde compraría este producto si estuviera disponible?

23 respuestas



¿Qué mejora le gustaría ver en este producto? (respuesta abierta)

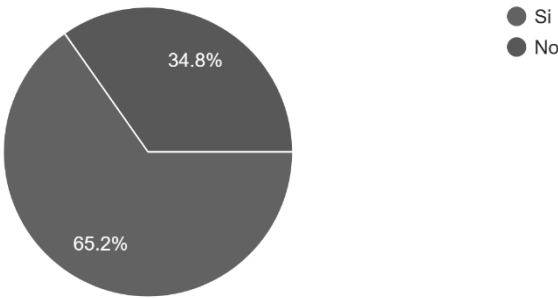
23 respuestas



MediCare+ (Dispensador de medicamentos).

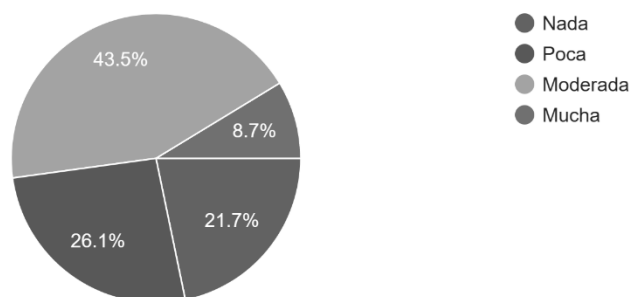
¿Alguna persona en su hogar toma medicamentos con regularidad?

23 respuestas



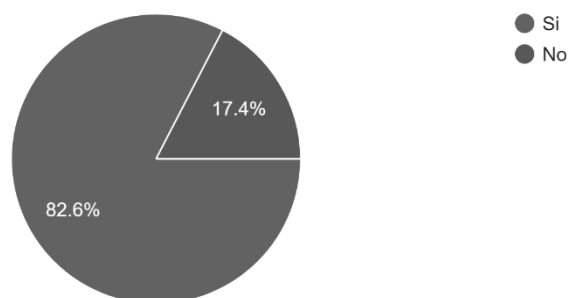
¿Con qué dificultad recuerdan tomar su medicamento a la hora correcta?

23 respuestas



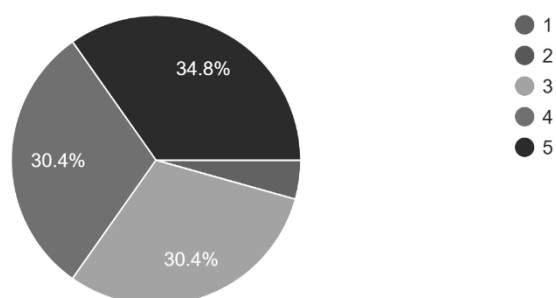
¿Le interesaría un dispensador que libere la pastilla automáticamente a la hora programada?

23 respuestas



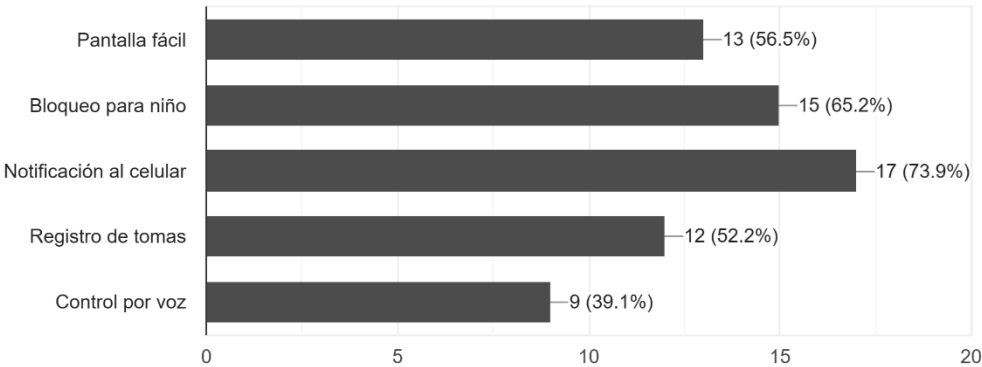
¿Cuánta confianza le daría a un dispositivo para dosificar medicamentos? donde 5 es el mas confiable

23 respuestas



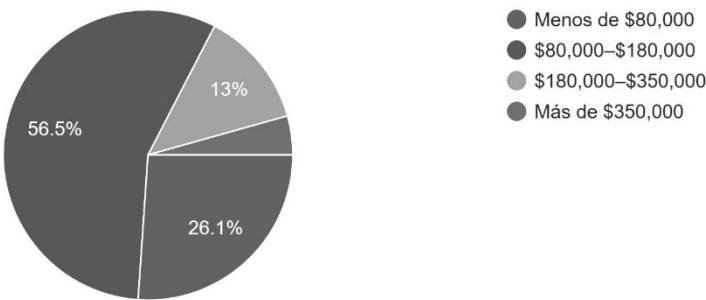
¿Qué funciones considera indispensables? (marcar hasta 3)

23 respuestas



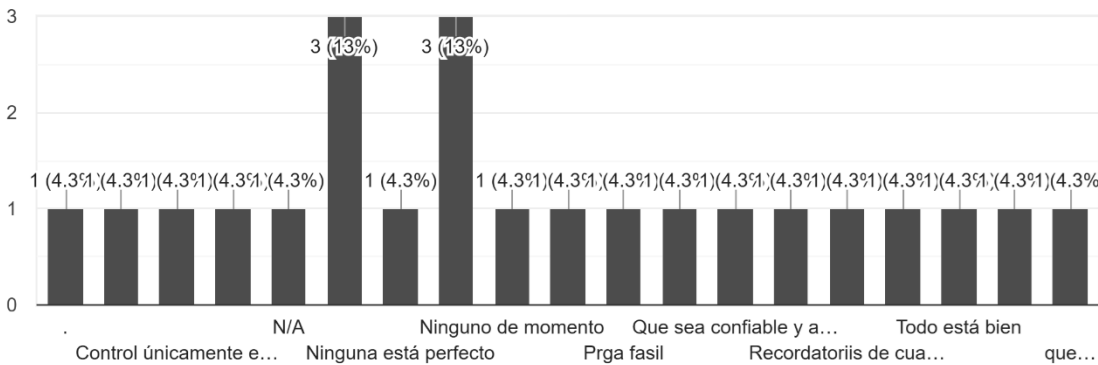
¿Cuánto estaría dispuesto a pagar?

23 respuestas



¿Qué cambio sugeriría para que lo comprara para su hogar? (abierta)

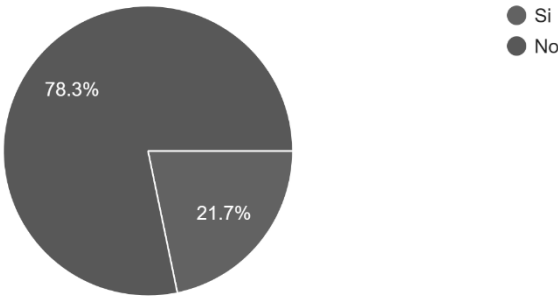
23 respuestas



CoolAir Auto (Ventilador automático por temperatura).

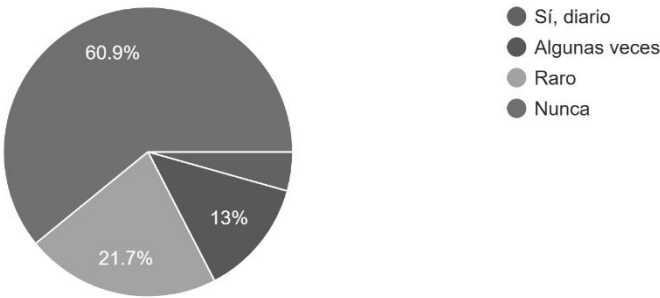
¿En su hogar sufren por altas temperaturas?

23 respuestas



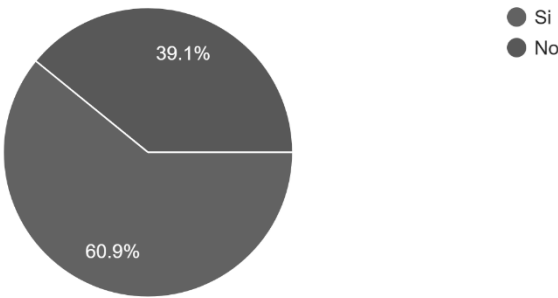
¿Usa ventilador con frecuencia?

23 respuestas

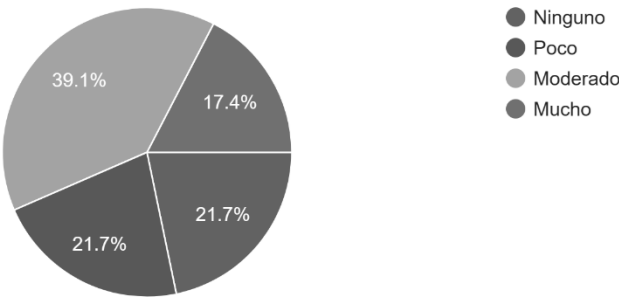


¿Consideraría comprar un ventilador que se enciende solo cuando la temperatura sube?

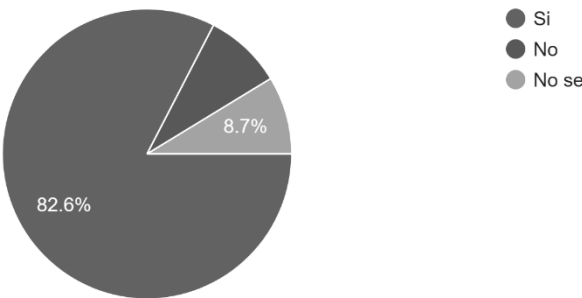
23 respuestas



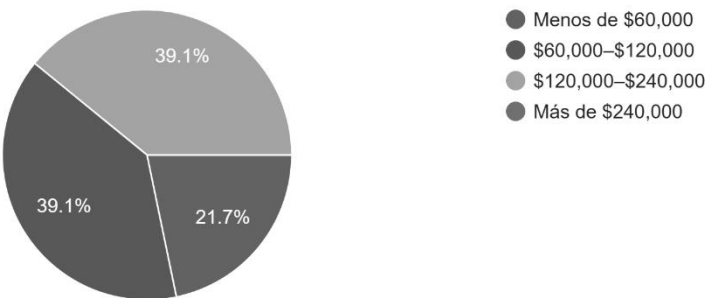
¿Qué ahorro mensual estaría dispuesto a aceptar como beneficio (p. ej. menor consumo de energía)
23 respuestas



¿Preferiría un ventilador con opción a energía solar
23 respuestas

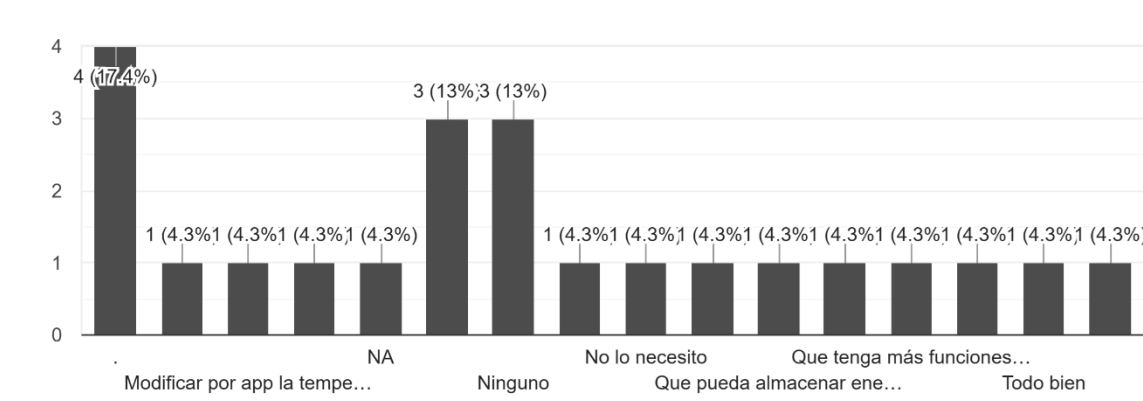


¿Cuánto pagaría por un ventilador inteligente (un solo equipo)?
23 respuestas



¿Qué mejora le haría más útil este ventilador? (abierta)

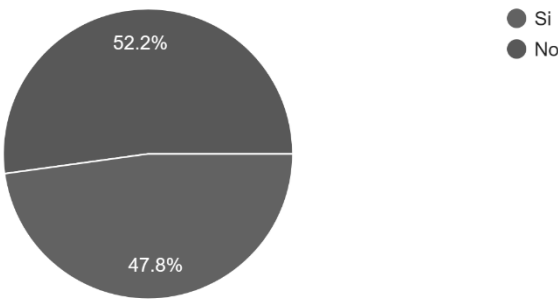
23 respuestas



SunTrack (Panel solar con seguidor).

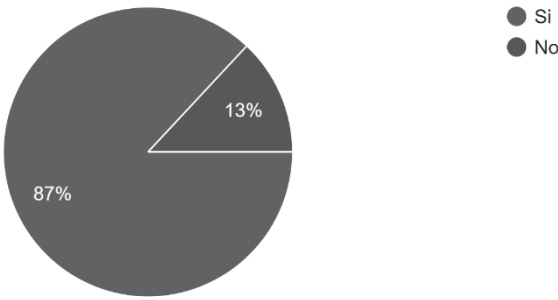
¿Tiene paneles solares o conoce a alguien que los tenga?

23 respuestas



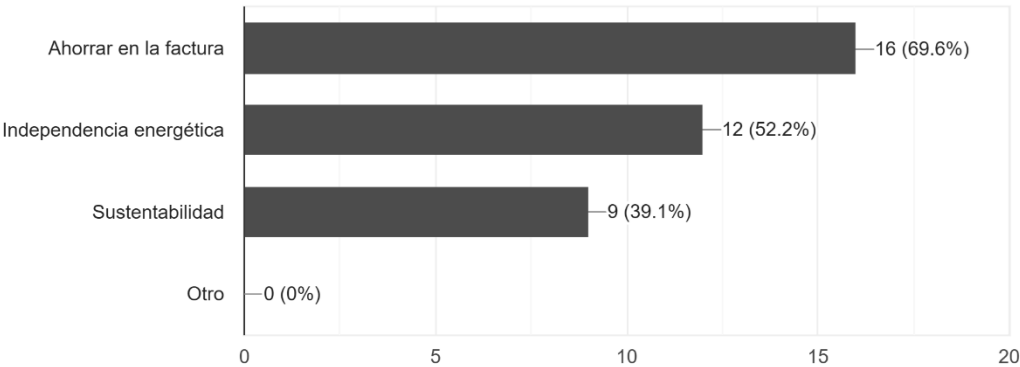
¿Conoce o le interesa la energía solar para reducir gastos de electricidad?

23 respuestas



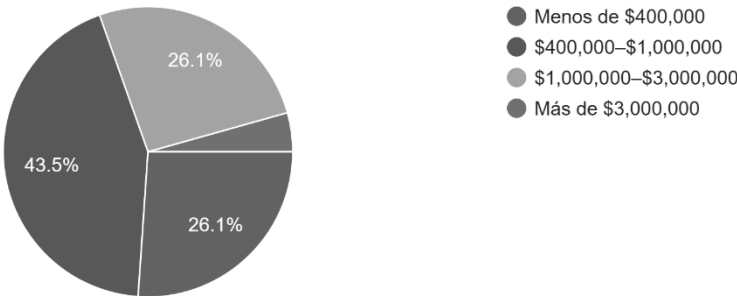
¿Cuál sería su principal motivación para instalarlo? (marcar una)

23 respuestas



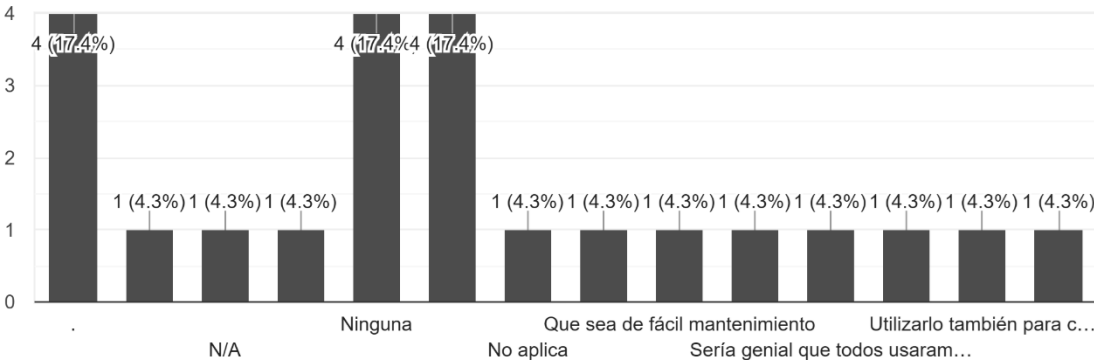
¿Cuánto estaría dispuesto a pagar adicional por el sistema de seguimiento (como inversión única)?

23 respuestas



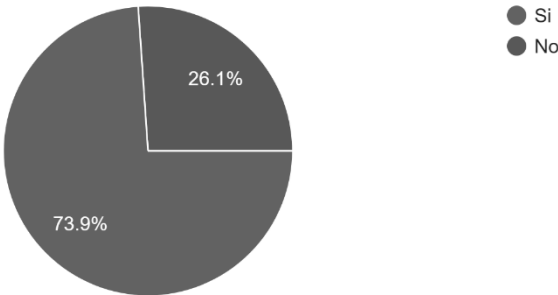
Comentarios o mejoras que usted sugiere (abierta).

23 respuestas

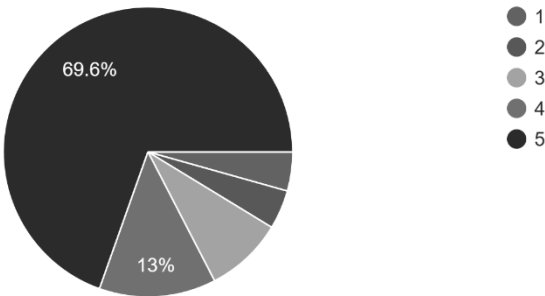


JumpStart 12V (Batería portátil de arranque).

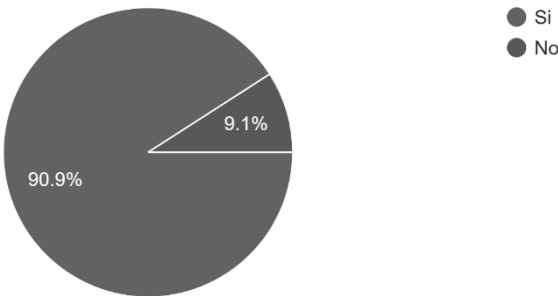
¿Ha quedado alguna vez varado por la batería del vehículo
23 respuestas



¿Qué tan útil le resultaría un dispositivo que arranca su carro sin ayuda de otro (1–5)
23 respuestas

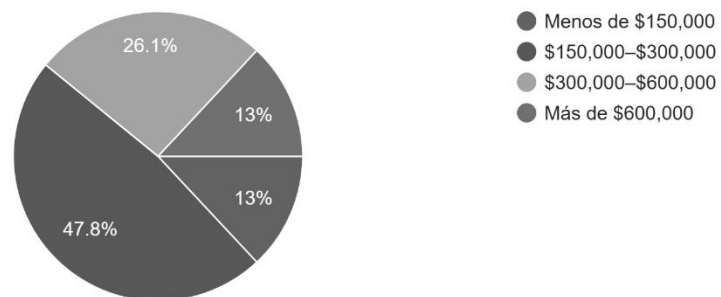


¿Preferiría que el dispositivo tenga linterna y puertos USB?
22 respuestas



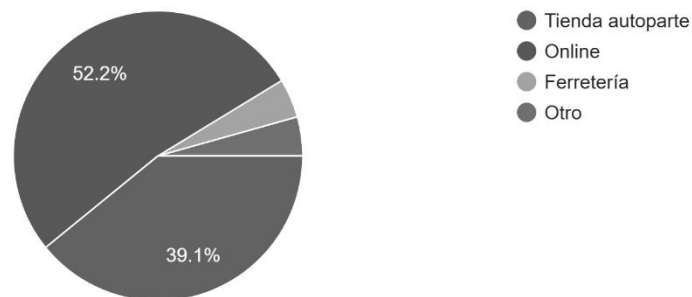
¿Cuánto estaría dispuesto a pagar por este dispositivo?

23 respuestas



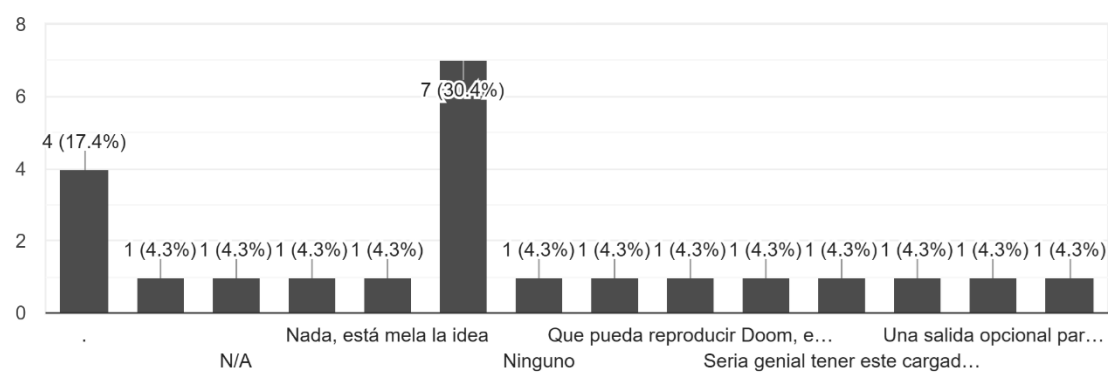
¿Dónde lo compraría?

23 respuestas



Sugerencias o mejoras (abierta).

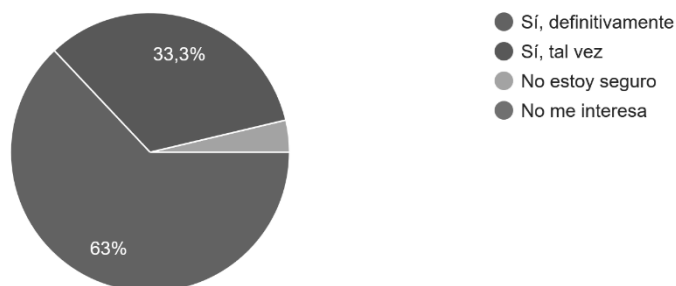
23 respuestas



SmartEyes (Gafas inteligentes).

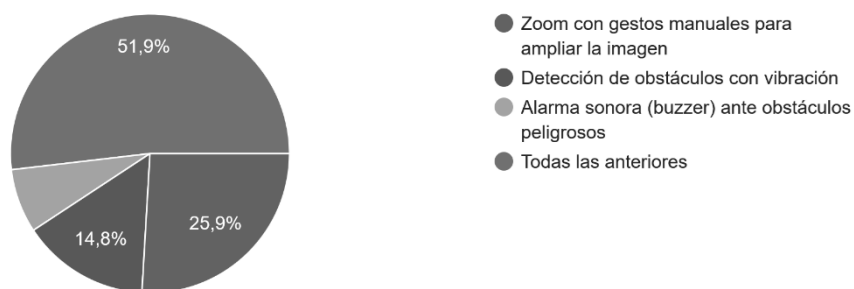
¿Le interesaría adquirir unas gafas inteligentes como "SmartEyes", que mejoran la visión y detectan obstáculos?

27 respuestas



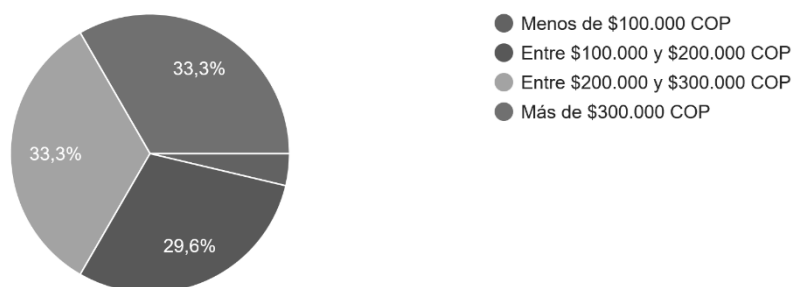
¿Qué función considera más importante en las "SmartEyes"?

27 respuestas



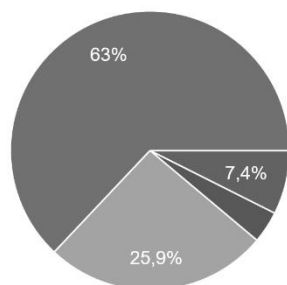
¿Qué rango de precio estaría dispuesto(a) a pagar por este producto?

27 respuestas



¿Preferiría que el aviso de obstáculos sea...?

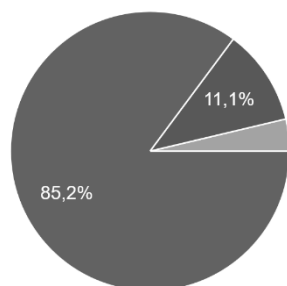
27 respuestas



- Solo con vibración
- Solo con sonido (buzzer)
- Ambos (vibración + sonido)
- Depende de la situación (opción para desactivar sonido o vibración)

¿Qué tan importante considera que las gafas tengan diseño liviano y estético?

27 respuestas

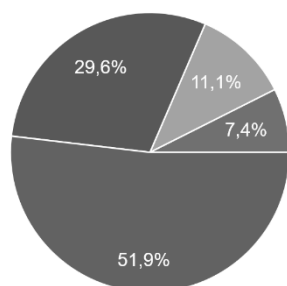


- Muy importante
- Importante
- Poco importante
- No es importante

GoodClothing (Closet inteligente con gestión de prendas).

¿Le interesaría adquirir un Closet Inteligente que organice y sugiera combinaciones de ropa automáticamente?

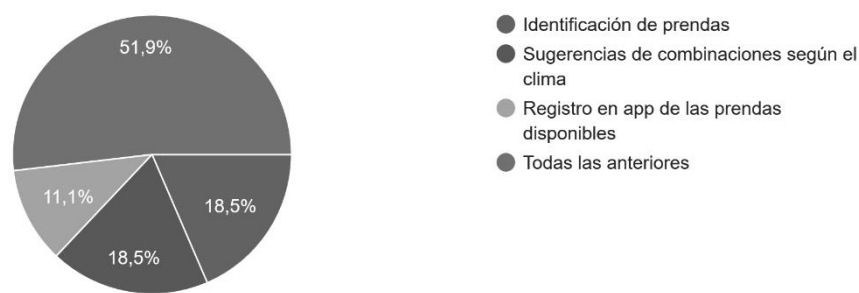
27 respuestas



- Sí, definitivamente
- Sí, tal vez
- No estoy seguro
- No me interesa

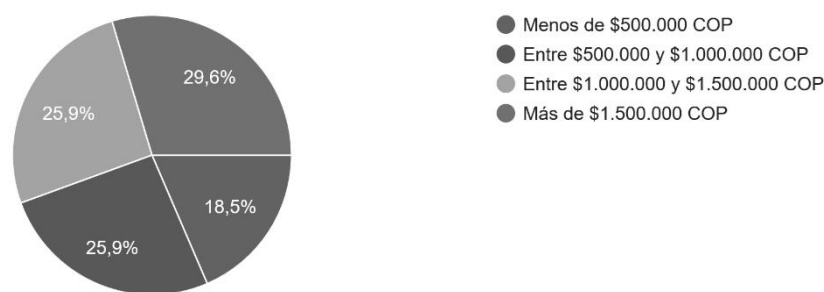
¿Qué característica le parece más útil en un Closet Inteligente?

27 respuestas



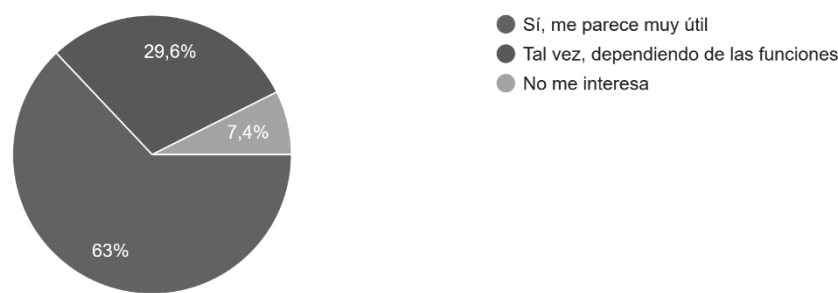
¿Qué rango de precio estaría dispuesto(a) a pagar por este producto?

27 respuestas



¿Le gustaría que el closet se conecte a una app para mostrar estadísticas de uso de su ropa?

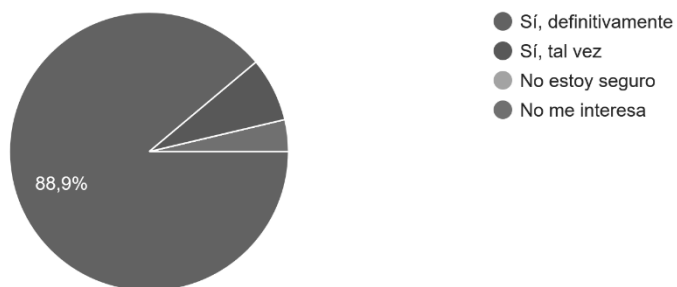
27 respuestas



NiceLife (Sistema de asistencia para personas mayores).

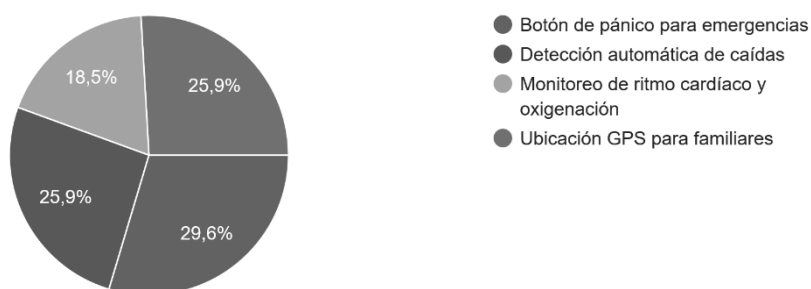
¿Considera útil un dispositivo portátil que ayude a personas mayores a pedir ayuda y que detecte caídas automáticamente?

27 respuestas



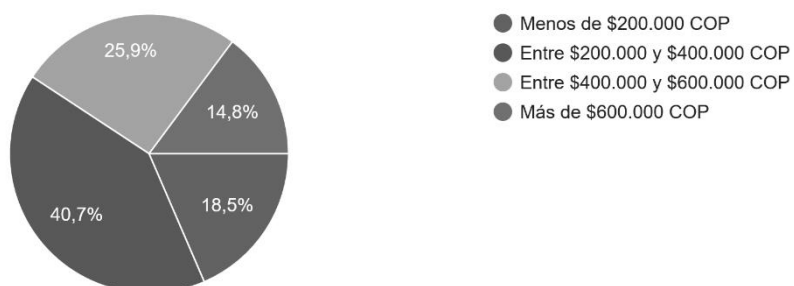
¿Qué función considera más importante en este sistema?

27 respuestas



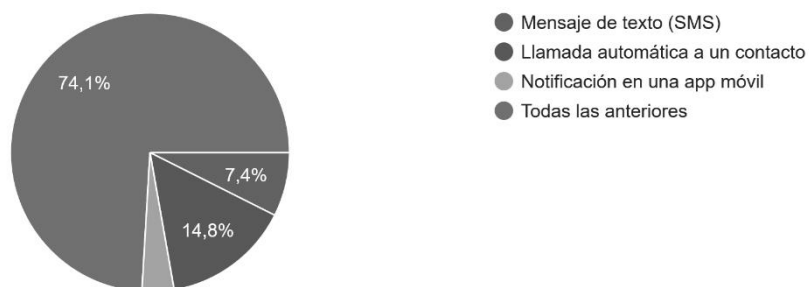
¿Qué rango de precio estaría dispuesto(a) a pagar por este producto?

27 respuestas



¿Cómo preferiría que el dispositivo envíe las alertas?

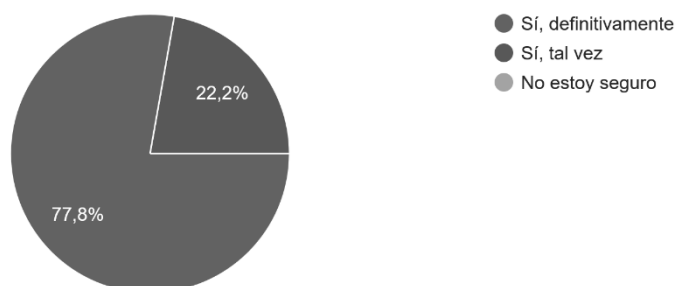
27 respuestas



Reciclajeficiente (Sistema de reciclaje inteligente).

¿Le interesaría un sistema que clasifique automáticamente la basura en plásticos, metales y orgánicos?

27 respuestas



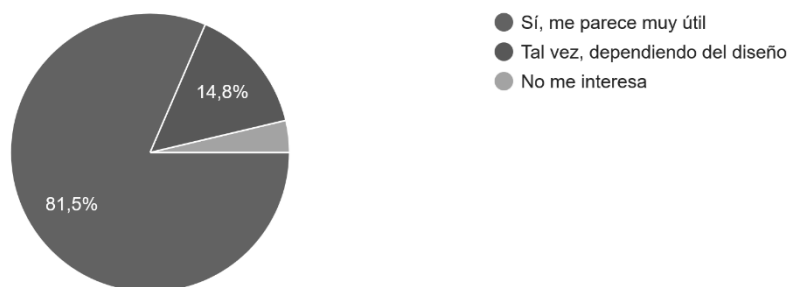
¿Dónde considera más útil instalar este sistema?

27 respuestas



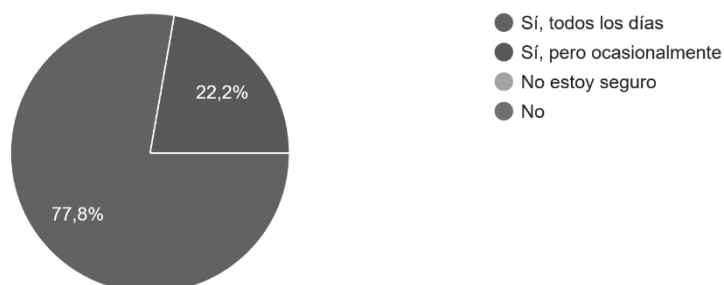
¿Le gustaría que el sistema muestre en una pantalla o app las estadísticas de reciclaje realizadas?

27 respuestas



¿Estaría dispuesto(a) a usar este sistema de forma frecuente para contribuir al medio ambiente?

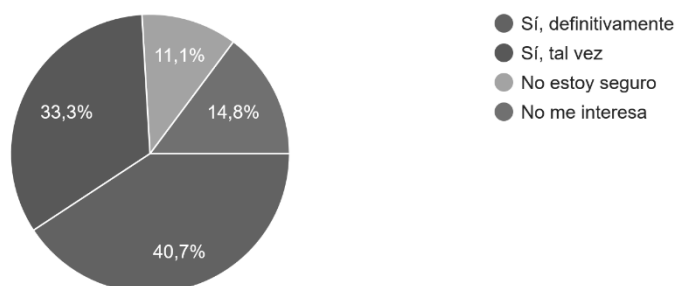
27 respuestas



HappyPlants (Invernadero autónomo con control de clima).

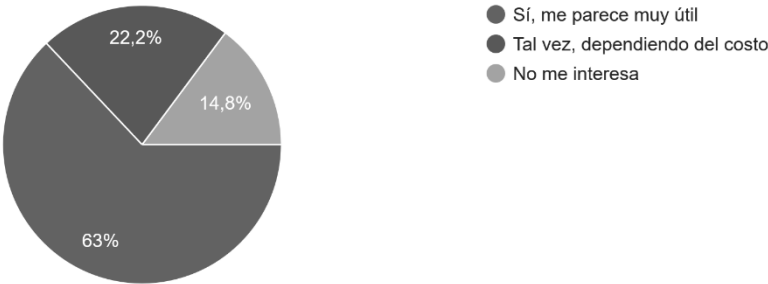
¿Le interesaría un sistema de invernadero autónomo que controle temperatura, humedad, luz y riego automáticamente?

27 respuestas



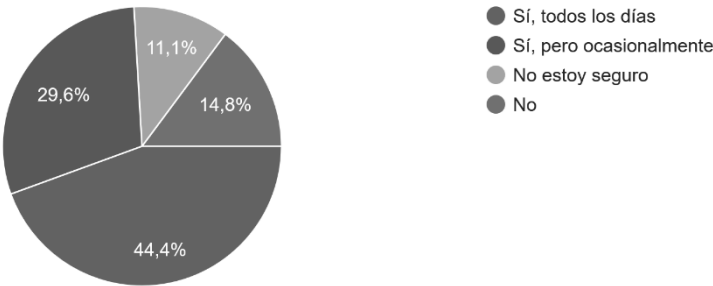
¿Le gustaría que “HappyPlants” guarde un historial y gráficos de crecimiento y condiciones ambientales?

27 respuestas



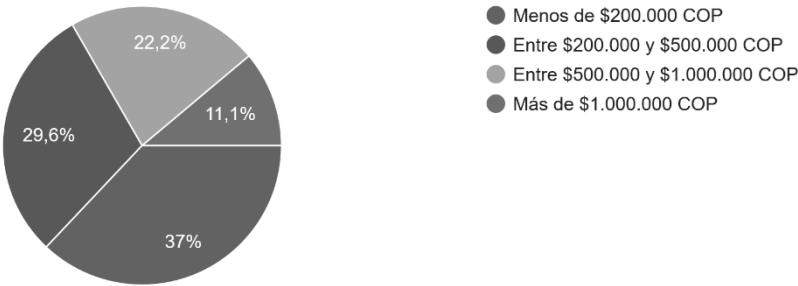
¿Estaría dispuesto(a) a usar este sistema de forma continua para optimizar el cuidado de sus plantas?

27 respuestas



¿Qué rango de precio estaría dispuesto(a) a pagar por este producto?

27 respuestas

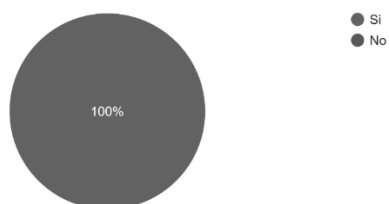


Selección de ideas

Sin embargo, para escoger dos ideas, para que alguno sea el candidato a producto a realizar. Se tomaron en cuenta las estadísticas correspondientes a preferencia de cada idea, midiendo el interés de la población en cada producto.

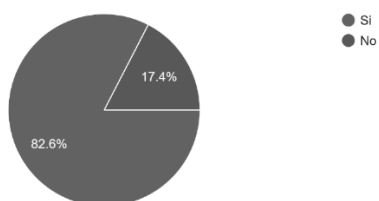
KeyFinder Pro (Buscador de objetos personales).

¿Le interesaría un dispositivo que suene y parpadee para ayudarle a encontrar objetos?
23 respuestas



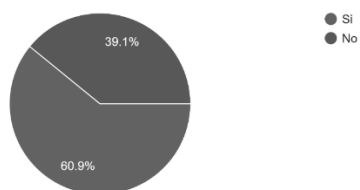
MediCare+ (Dispensador de medicamentos).

¿Le interesaría un dispensador que libere la pastilla automáticamente a la hora programada?
23 respuestas



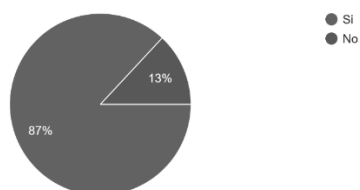
CoolAir Auto (Ventilador automático por temperatura).

¿Consideraría comprar un ventilador que se enciende solo cuando la temperatura sube?
23 respuestas



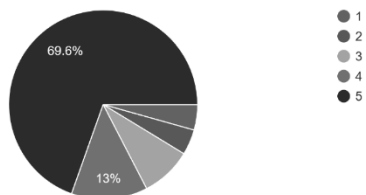
SunTrack (Panel solar con seguidor).

¿Conoce o le interesa la energía solar para reducir gastos de electricidad?
23 respuestas



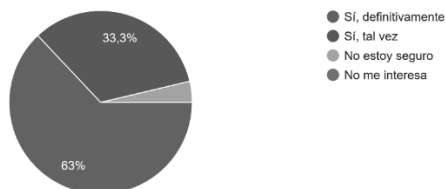
JumpStart 12V (Batería portátil de arranque).

¿Qué tan útil le resultaría un dispositivo que arranca su carro sin ayuda de otro (1-5)
23 respuestas



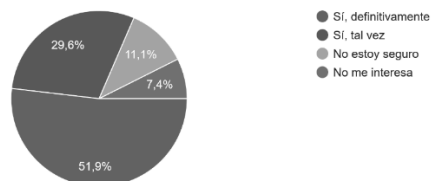
SmartEyes (Gafas inteligentes).

¿Le interesaría adquirir unas gafas inteligentes como "SmartEyes", que mejoran la visión y detectan obstáculos?
27 respuestas



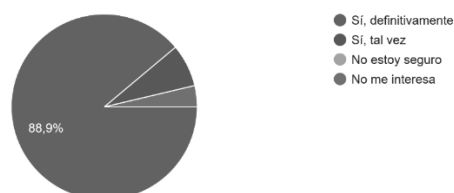
GoodClothing (Closet inteligente con gestión de prendas).

¿Le interesaría adquirir un Closet Inteligente que organice y sugiera combinaciones de ropa automáticamente?
27 respuestas



NiceLife (Sistema de asistencia para personas mayores).

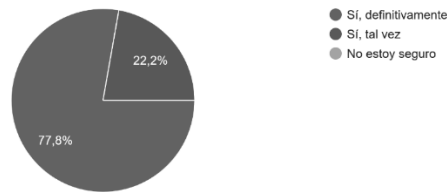
¿Considera útil un dispositivo portátil que ayude a personas mayores a pedir ayuda y que detecte caídas automáticamente?
27 respuestas



Reciclajeficiente (Sistema de reciclaje inteligente).

¿Le interesaría un sistema que clasifique automáticamente la basura en plásticos, metales y orgánicos?

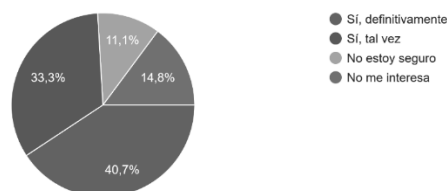
27 respuestas



HappyPlants (Invernadero autónomo con control de clima).

¿Le interesaría un sistema de invernadero autónomo que controle temperatura, humedad, luz y riego automáticamente?

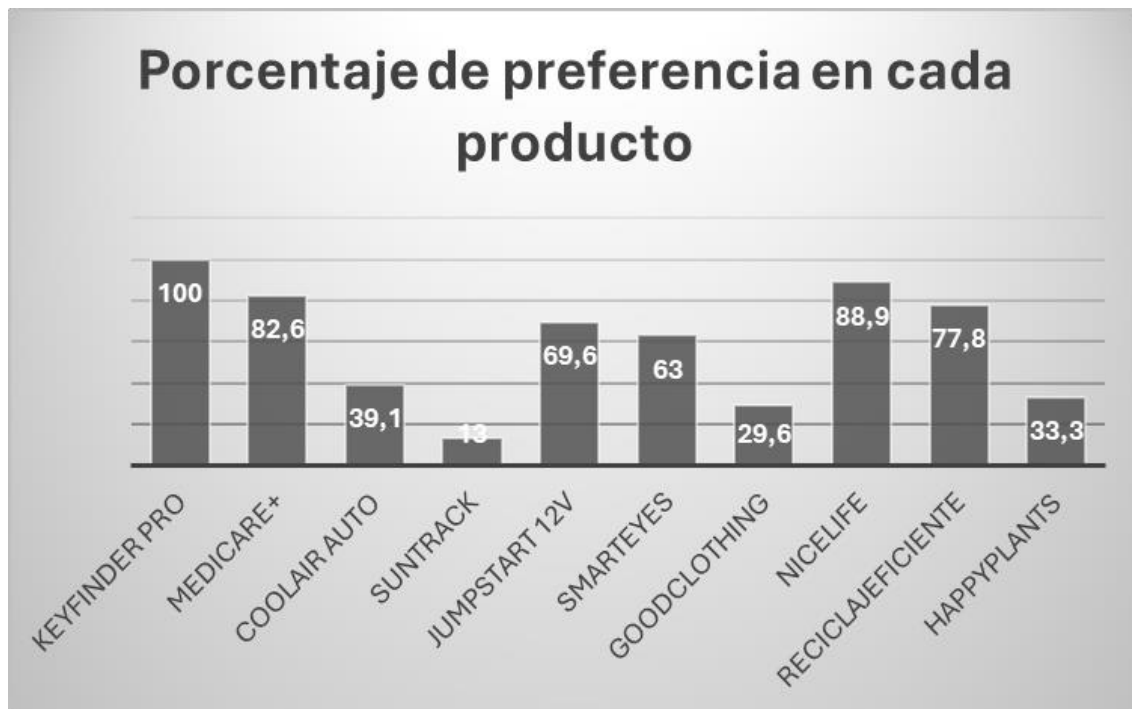
27 respuestas



Reuniendo todas las estadísticas, podemos realizar un listado de las ideas entre mayor y menor preferencia que sería el siguiente:

1. KeyFinder Pro – 100% (Dispositivo para encontrar objetos personales)
2. NiceLife – 88.9% (Asistente para personas mayores con detección de caídas)
3. MediCare+ – 82.6% (Dispensador automático de medicamentos)
4. Reciclajeficiente – 77.8% (Sistema de reciclaje inteligente)
5. JumpStart 12V – 69.6% (Batería portátil para arranque de autos)
6. SmartEyes – 63% (Gafas inteligentes para visión y detección de obstáculos)
7. CoolAir Auto – 39.1% (Ventilador automático por temperatura)
8. HappyPlants – 33.3% (Invernadero autónomo con control climático)
9. GoodClothing – 29.6% (Closet inteligente con sugerencias de outfits)
10. SunTrack – 13% (Panel solar con seguidor para ahorro energético)

Y esto se puede visualizar en el siguiente gráfico.



Con esto podemos concluir que:

- KeyFinder Pro es el más demandado (aceptación unánime).
- NiceLife y MediCare+ destacan en soluciones de salud y asistencia.
- SunTrack y GoodClothing tienen la menor preferencia, posiblemente por nichos más específicos.

Capítulo 3

Se debió tener en cuenta que las ideas propuestas no se hayan realizado anteriormente en el curso, o que, si ya hay un proyecto existente de esa idea, entonces se debería de agregar alguna mejora. Preferiblemente se debería de escoger alguna idea que no se haya trabajado. Por ello no se escogió la propuesta que quedo en primer lugar (KeyFinder Pro), entonces se tuvo que indagar acerca de los siguientes puestos de la lista, cercanos al primer lugar y que se cumpla con lo anteriormente mencionado. Finalizando el ejercicio se decidió escoger las propuestas: **Reciclajeficiente** (Sistema de reciclaje inteligente), y **SmartEyes** (Gafas inteligentes).

Realización de Requerimientos

Lo siguiente a realizar, es el clasificación y redacción de los requerimientos funcionales y no funcionales de las dos propuestas escogidas, esto para revisar y seleccionar cual es el más apto a desarrollar, dependiendo de cuantos requerimientos debe cumplir y su dificultad. En este caso los requerimientos de cada uno son los siguientes:

Requerimientos proyecto Reciclajeficiente (Sistema de reciclaje inteligente)

Funcionales	No Funcionales
Avisar al estar llena	Adaptable a diferentes espacios (hogar, industria, universidades, centros comerciales)
Manejo de residuos diferenciados (clasificación: papel, cartón, botellas, orgánicos, metal)	De uso asequible (costo accesible para el público)
Que pueda vaciarse fácil (compartimientos accesibles)	Escalable a centros de acopio (uso industrial)
Proyección de videos educativos sobre reciclaje	Resistente, higiénica y segura
Iluminación para visibilidad en lugares oscuros	Eficiencia energética (panel solar o bajo consumo)
Cámara con IA	Contribución al medio ambiente
Pantalla con estadísticas de reciclaje realizada en la caneca	Resistente al agua

Requerimientos proyecto SmartEyes (Gafas inteligentes)

Funcionales	No Funcionales
Traducción en tiempo real (texto o voz)	Diseño liviano, cómodo y discreto
Notificaciones conectadas vía Bluetooth	Estilo variado (no muy tecnológico a la vista)
Cámara integrada con opción de grabar videos y conexión al celular	Recargables (batería de larga duración)
Localización de las gafas (alarma o luz al activarse por voz)	Materiales resistentes
Protección UV, filtros de luz y transición fotocromática	Opciones de personalización (color, estilo, modelos)
Dictado de hora por voz	
Despertar a conductor con somnolencia con vibración o alarma	

Actividad en clase 1

Se realizó un código en **lenguaje C** en la plataforma **platform.io** dentro de la aplicación **VisualStudio**, esto para familiarizarse con el entorno al momento de realizar el código para el proyecto seleccionado. Este se basó en el control simple del integrado al módulo **ESP-WROOM-32**.

Su código es el siguiente (También dentro de este repertorio está el archivo .txt con el mismo nombre):

[Código de parpadeo de LED integrado.txt](#)

```
extern "C" void app_main() ;

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "sdkconfig.h"

#define LED_PIN GPIO_NUM_2 // LED en el pin 2 (en muchas placas es el LED
integrado)

void app_main(void)
{
    // Configurar pin como salida
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

    while (1) {
        gpio_set_level(LED_PIN, 1); // Encender LED pin 2
        vTaskDelay(1000 / portTICK_PERIOD_MS); // 1 segundo
        gpio_set_level(LED_PIN, 0); // Apagar LED pin 2
        vTaskDelay(1000 / portTICK_PERIOD_MS); // 1 segundo
    }
}
```

Capítulo 4

Se socializaron los requerimientos y con ellos se procedió a realizar un análisis DOFA (Debilidades, Oportunidades, Fortalezas y Amenazas), para identificar factores internos y externos para tomar decisiones más eficaces y planear estrategias en el proyecto, además de visualizar cual es más viable a realizar.

Análisis DOFA

DOFA proyecto Reciclajeficiente (Sistema de reciclaje inteligente)

Fortalezas	Oportunidades	Debilidades	Amenazas
Promueve el reciclaje con incentivos → impacto social y ambiental positivo.	Alta demanda en programas de reciclaje y economía circular.	Requiere mantenimiento (autolimpieza, drenaje de líquidos).	Competencia con sistemas tradicionales de reciclaje (económicos y simples).
Fácil de justificar en proyectos de sostenibilidad, universidades, empresas y ciudades inteligentes.	Posible financiamiento gubernamental o municipal.	Posible complejidad mecánica al integrar diferentes sensores y compuertas.	Normativas ambientales que podrían limitar algunos componentes (ej. cámaras con IA por privacidad).
Puede incluir sensores relativamente económicos (peso, nivel, humedad, inductivos).	Integración en proyectos de ciudades sostenibles.	Riesgo de vandalismo o mal uso en espacios públicos.	Necesidad de robustez alta en exteriores (clima, golpes, humedad).
Uso masivo en espacios públicos (gran alcance de usuarios).	Alianzas con empresas de reciclaje o de aseo.	Costo inicial puede ser elevado para hogares.	

DOFA proyecto SmartEyes (Gafas inteligentes)

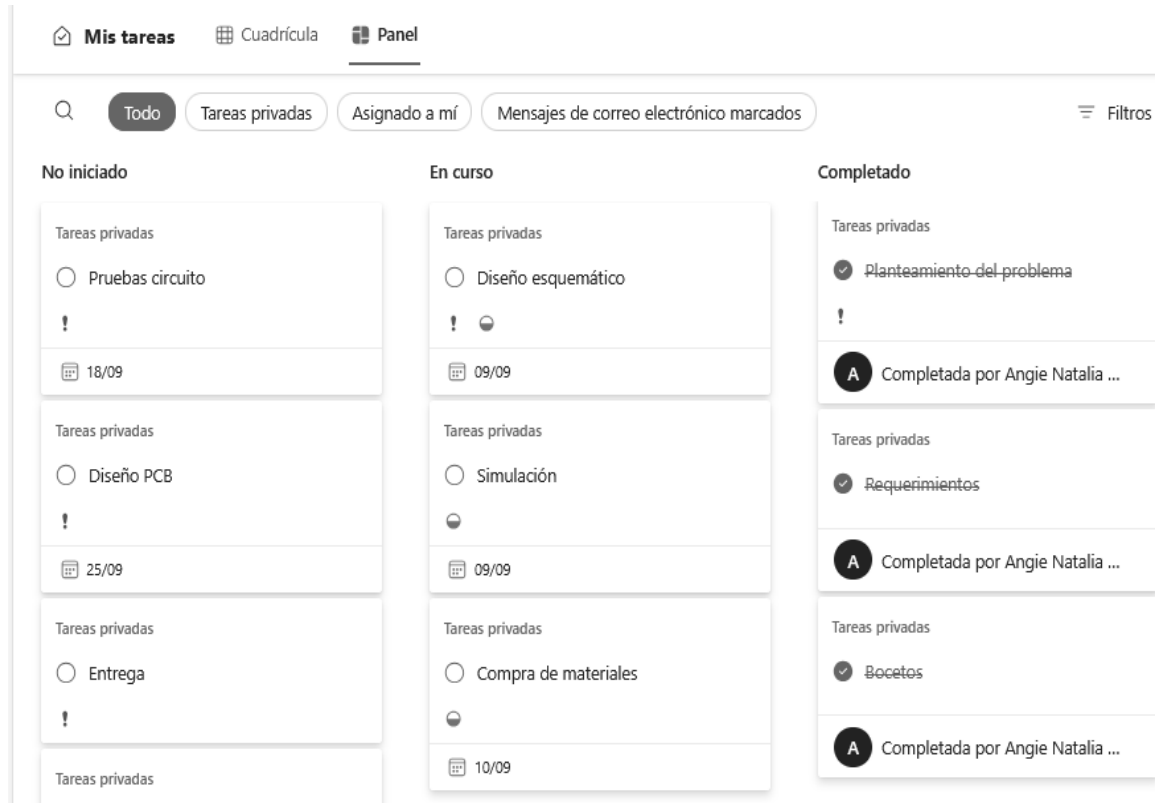
Fortalezas	Oportunidades	Debilidades	Amenazas
Innovación tecnológica llamativa (traducción, notificaciones, IA, cámara).	Creciente mercado de wearables y realidad aumentada.	Complejidad técnica muy alta (miniaturización, batería, conectividad, óptica AR).	Alta expectativa del usuario (puede compararse con grandes marcas).
Diferenciación en el mercado frente a gafas convencionales.	Interés en dispositivos de traducción y conectividad.	Requiere inversión en hardware especializado (microproyector, ópticas, procesador).	Riesgos de privacidad (cámara en espacios públicos).
Personalización estética (colores, modelos, estilos).	Posible integración con asistentes virtuales existentes (Google, Alexa, etc.).	Alto costo de producción → difícil de hacer accesible en prototipo.	Regulaciones en algunos países sobre uso de gafas con cámara.
Potencial para aplicaciones en turismo, educación y salud.	Tendencia hacia moda tecnológica (fashion tech).	Competencia fuerte (Google Glass, Meta, Xiaomi, etc.).	Posible rechazo si el diseño no es estético o es muy pesado.

Gracias a este análisis podemos concluir que el proyecto más viable a corto plazo es el de **Reciclajeficiente (Sistema de reciclaje inteligente)** por las presentes razones:

- Usa tecnología alcanzable (sensores accesibles, microcontroladores económicos).
- Tiene fuerte impacto social y ambiental.
- Es escalable desde un prototipo pequeño hasta un modelo industrial.
- Más fácil de justificar en convocatorias de innovación sostenible.

Organizar planeación de actividades a realizar para el proyecto

Con la propuesta ya escogida, se procede a crear un cronograma de actividades con la aplicación **Planner** de Microsoft, el cual, nos permite darles un orden a las tareas, incluyendo el calendario, importancia de la actividad e indicar si se está desarrollando o ya se cumplió.



Las actividades propuestas fueron las siguientes:

1. Planteamiento del problema.
2. Requerimientos.
3. Bocetos.
4. Diseño esquemático.
5. Simulación.
6. Compra de materiales.
7. Pruebas circuito.
8. Diseño PCB.
9. Impresión de PCB.
10. Ensamble.

11. Construcción con circuito.
12. Pruebas finales.
13. Entrega.

Temática dictada en Clase:

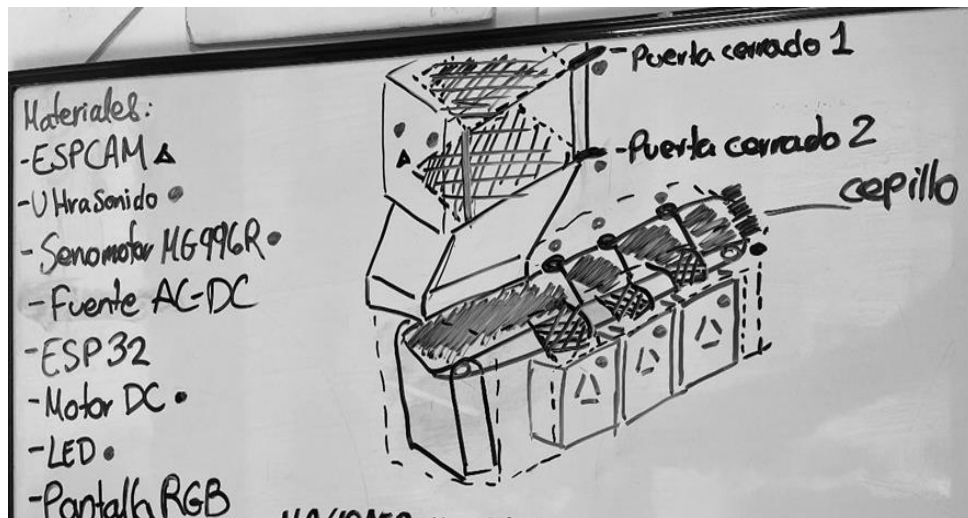
Se hablo acerca de la teoría del valor ganado (Earned Value Management (EVM)), la cual contiene la línea base de tiempo, la línea base de costos, además de tener en cuenta el alcance; esta teoría permite medir el rendimiento y el progreso de un proyecto, importante cuando se debe de tomar decisiones correctivas cuando se detectan desviaciones en el proceso. Este contiene:

- **Valor Planificado (PV):** Valor del trabajo que se esperaba haber completado.
- **Valor Ganado (EV):** Valor del trabajo completado hasta la fecha.
- **Costo Real (AC):** El costo real incurrido para completar el trabajo hasta la fecha.

También se habló de que se debe de tener en cuenta que la calidad de un proyecto nunca se negocia, pero el tiempo y el costo sí.

Capítulo 5

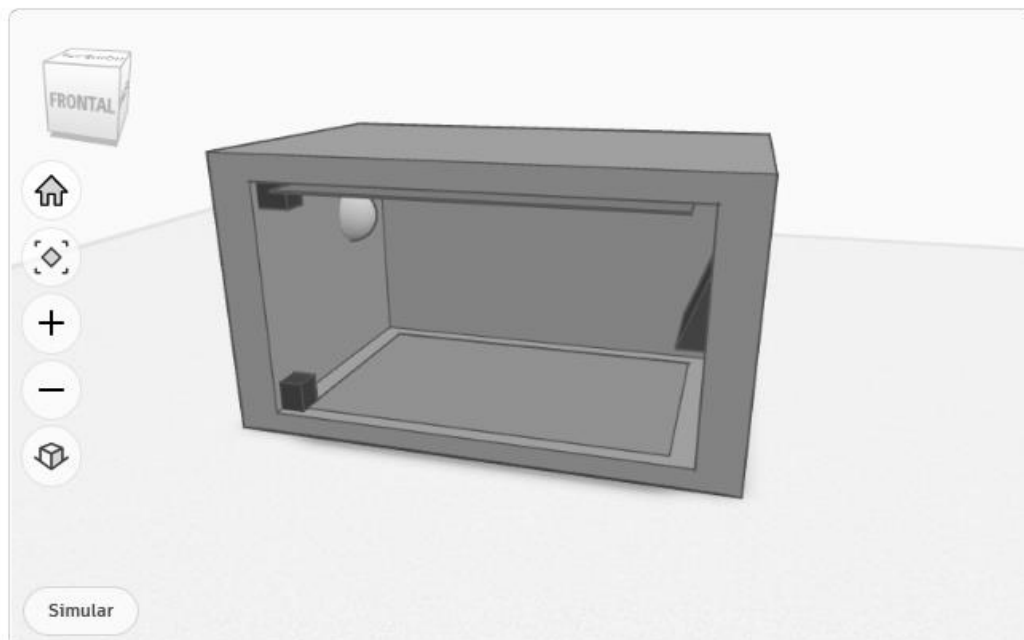
Se realizó un boceto donde se puede visualizar la arquitectura física que puede tener el proyecto:



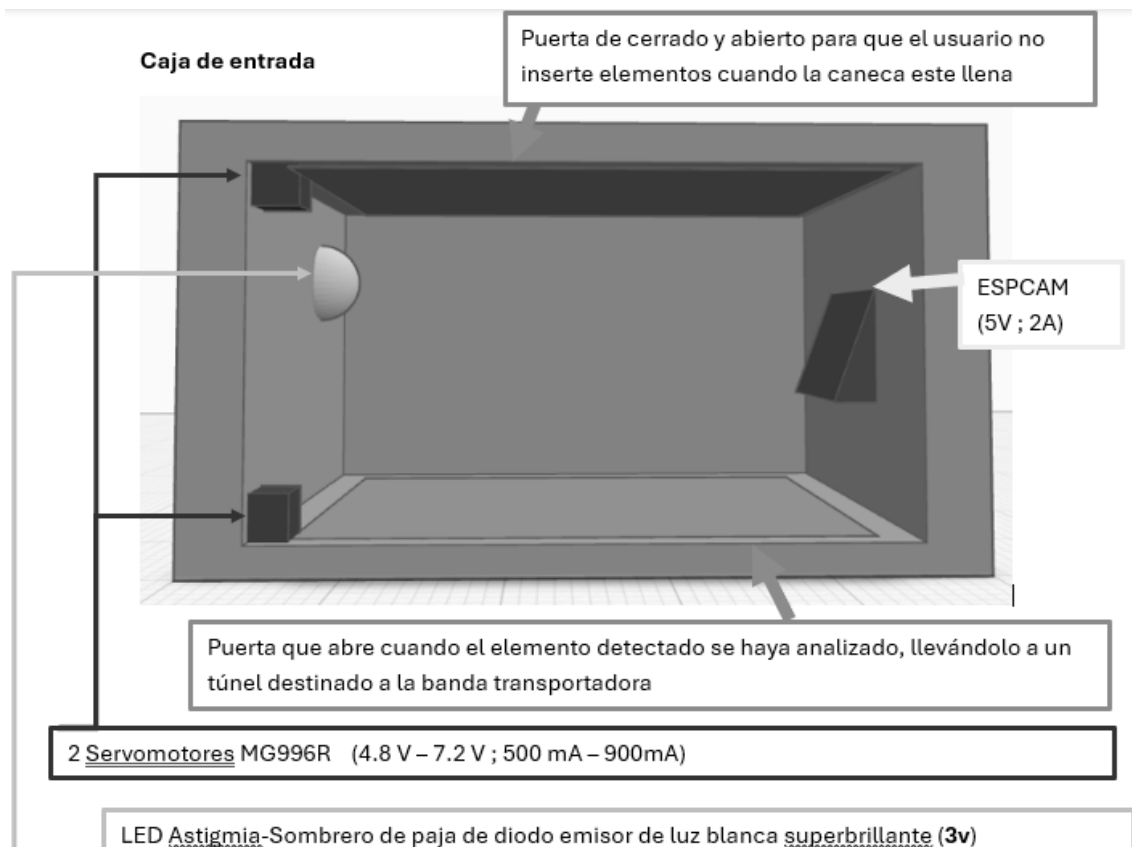
Y un diseño en 3D donde se ubicaría la ESPCAM para el análisis con visión inteligente del objeto desechado.

En la plataforma de tinkercad:

Caja proyecto diseño 2



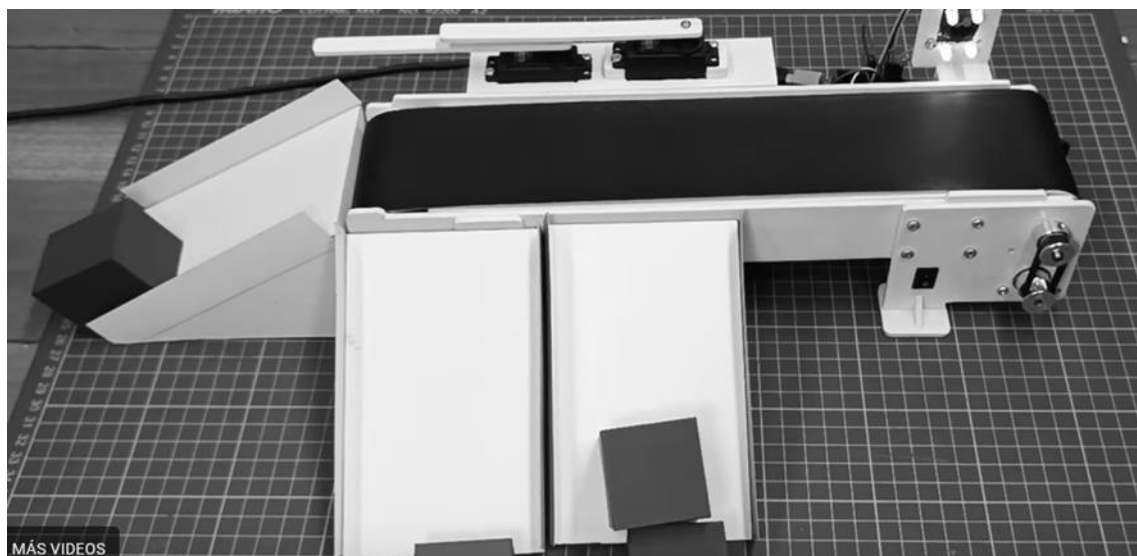
Señalamiento de piezas:



También el diseño de una banda transportadora para la debida clasificación de los productos se debió a la inspiración del siguiente video realizado por "Construye Fácil", titulado "Como hacer una Cinta Transportadora (Planos PDF)" y subido en la plataforma de Youtube.

Enlace al video: <https://www.youtube.com/watch?v=PjL9drjZAQU>

Imagen de referencia:



Desde aquí se comenzó a planear el diseño de arquitectura que va a tener el proyecto, teniendo en cuenta los capítulos anteriores del presente documento. Para ello se realizó una lista en el software Microsoft Excel con los componentes necesarios, los proveedores, sus precios y sus datasheet (para luego realizar la suma de potencias que se necesiten en cada componente). El documento se puede visualizar aquí:

[Tabla de productos y proveedores.xlsx](#)

La vista a la tabla es de la siguiente manera:

1	Componente	Proveedor 1	Precio 1	Link 1	Proveedor 2	Precio 2	Link 2	Proveedor 3	Precio 3	Link 3	Modelo y Datasheet
2	4 x Servomotores de piñones de plástico	Aliexpress	4877,32	Link producto	Sigma	9996	Link producto	Zamux	9500	Link producto	SG90
3	Servomotor de piñones metálicos	Aliexpress	14643	Link producto	Sigma	27965	Link producto	Electronilab	27900	Link producto	MG995
4	4 x Módulos sensor Ultrasonido	Electronilab	6100	Link producto	Sigma	6545	Link producto	Zamux	5900	Link producto	HC-SR04
5	3 x LED's Rojos	Electronilab	100	Link producto	Sigma	179	Link producto	Zamux /100 unidades	12000	Link producto	LED Rojo
6	7 x LED's superbrillantes Blancas	Aliexpress /100 unidades	9172,38	Link producto	Stereon	3000	Link producto	Temu /50 unidades	6117	Link producto	LED Blanco (Ver en deta
7	Matriz de LED 8x32 MAX7219	Aliexpress	15942,84	Link producto	Dynamo	30000	Link producto	Electronilab	29900	Link producto	Matriz 1088AS
8	ESP32-CAM	Aliexpress	35080,32	Link producto	Sigma	47600	Link producto	Zamux	38900	Link producto	ESP-CAM
9	Modulo de programación para ESPCAM	Aliexpress	7362,03	Link producto	Sigma	8925	Link producto	Zamux	5900	Link producto	ESP-CAM-MB
10	ESP32	Aliexpress	9907,24	Link producto	Sigma	22015	Link producto	Mactronica	17000	Link producto	ESP32-WROOM-32
11	DRV8825	Vistronica	6664	Link producto	Sigma	9996	Link producto	Zamux	13000	Link producto	DRV8825
12	LM2596	Aliexpress /10 unidades	8689,98	Link producto	Sigma	4998	Link producto	Electronilab	6500	Link producto	LM2596
13	LM1117DT-3.3	Aliexpress /10 unidades	5846,72	Link producto	Sigma	2142	Link producto	Electronilab /AZ1117	1100	Link producto	LM1117
14	BMS	Movitrionics	12600	Link producto	Zamux	11900	Link producto	Electronilab /40A	15900	Link producto	BMS 3S 10A 18650
15	NEMA-17	I+D Electronica	58429	Link producto	Zamux	65000	Link producto	Electronilab	55900	Link producto	Stepper Motor NEMA

Diagrama de arquitectura

Realizamos el siguiente diagrama de arquitectura, donde podemos visualizar cuales son los componentes y periféricos correspondientes a las entradas y salidas del sistema, y como todo se conecta entre sí en la unidad de control, que en este caso es la ESP32, para que durante el proceso se muestren los resultados en una pantalla de matriz LED, haciendo así una interacción hacia el usuario.

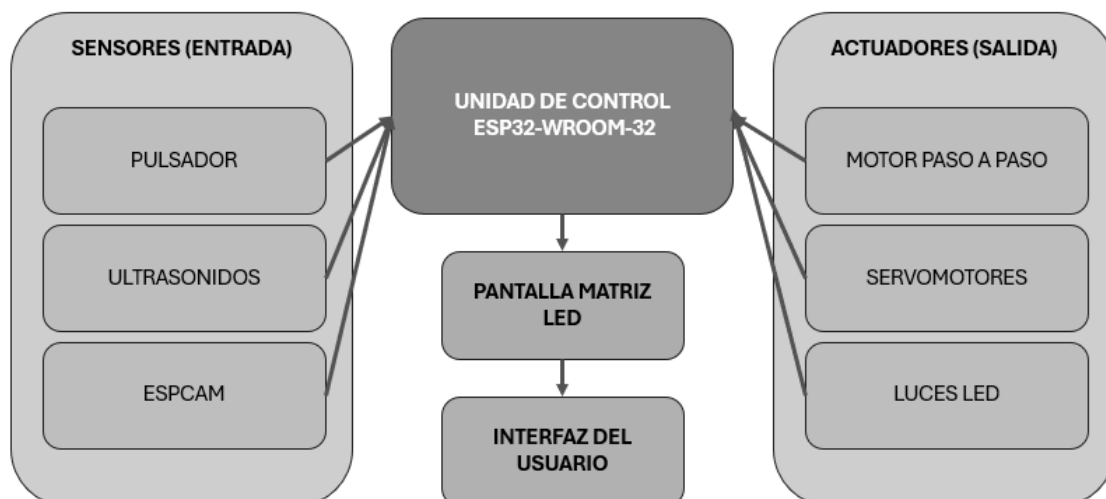
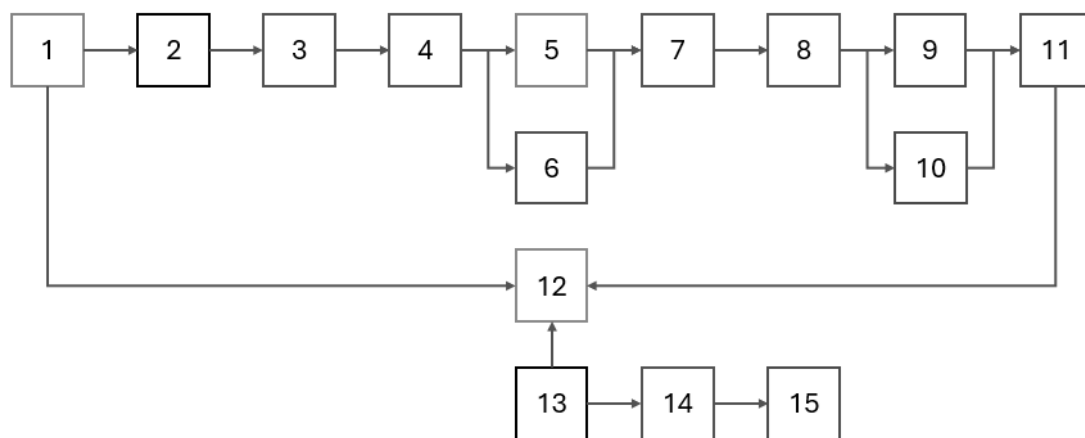


Diagrama de bloques de funcionamiento

Con lo anteriormente mencionado, procedemos a realizar el diagrama de bloques del funcionamiento con los pasos a continuación.

1. Encendido sistema con pulsador (encendido pantalla, ESP32, ESPCAM)
2. Entrada objeto a la caneca principal.
3. Reconocimiento Ultrasonido 1 a la entrada de la caneca principal
4. Envío información a ESP32.
5. Encendido diodos LED superbrillantes.
6. Reconocimiento ESPCAM.
7. Envío información a ESP32-WROOM-32 vía ESPNOW.
8. Envío información de ESP32 a servomotores y a ESPCAM.
9. Apertura servomotor 2, dejar caer objeto a banda transportadora.
10. Inicio recorrido motor paso a paso NEMA17.
11. Empuje de servomotor (3,4,5) al objeto hacia caneca correspondiente.
12. Mostrar en pantalla (nivel batería, mensaje de información de objeto y caneca, Alerta de llenado).
13. En caso de llenado de alguna caneca, mediante el reconocimiento del sensor ultrasonido (2,3,4).
14. Envío de información a ESP32.
15. Se cierra la compuerta de entrada con el servomotor 1.

Por lo cual el proceso de funcionamiento sería el siguiente.



Capítulo 6

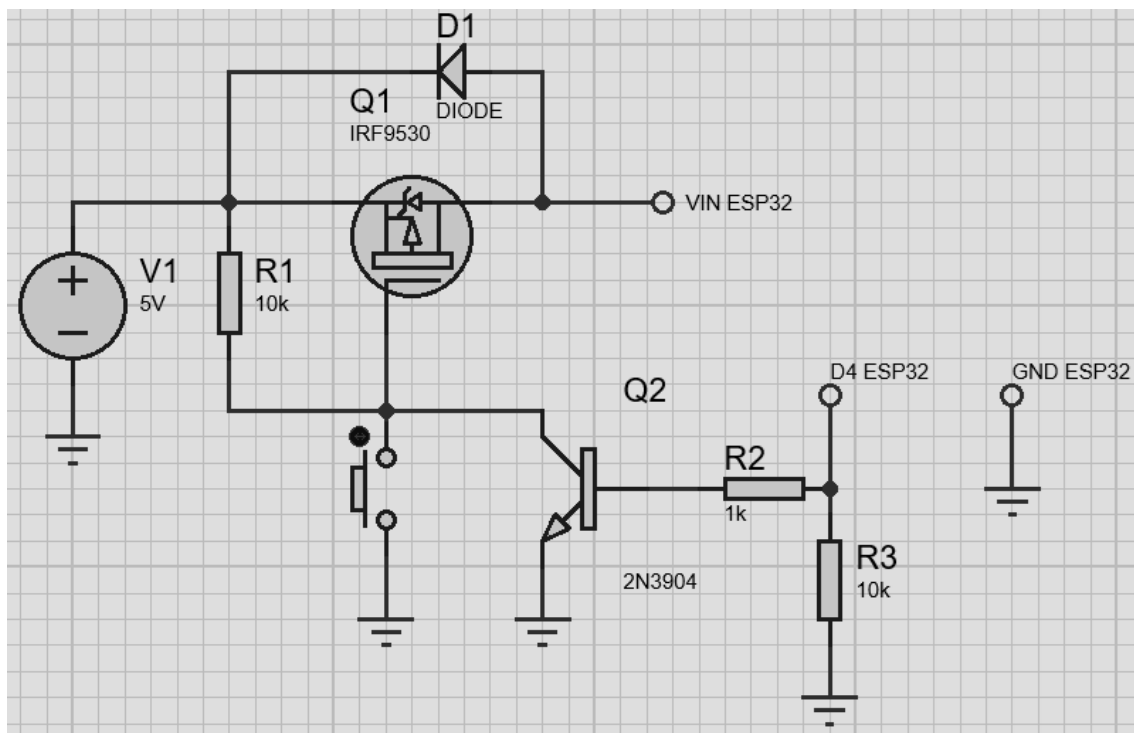
Actividad en clase 2:

En la **Actividad en clase 1** realizada en el **Capítulo 3**, se editó el código y el esquemático del circuito, para que su nuevo objetivo fuera el de presionar un pulsador, y que luego en 10 segundos se apague el sistema por completo, aun estando conectado a la fuente.

Para esta propuesta se usaron los materiales a continuación:

- Fuente externa de 5V.
- 2 x Resistencias 10 KOhm.
- 1 x Resistencia 1 KOhm.
- Pulsador.
- Transistor NPN 2N3904.
- Diodo 1N4007.
- MOSFET CANAL P IRF9530.
- Módulo ESP32.

El esquemático propuesto fue el siguiente.



El código que se programó fue "CODIGO ONOFF SISTEMA.txt" el cual se encuentra en el directorio del repositorio también.

CODIGO 2 ONOFF SISTEMA.txt

```
// CODIGO ENCENDER SISTEMA CON UN PULSADOR Y QUE SE APAGUE EN 10
SEGUNDOS (CON UN LED PARPADEANDO)
extern "C" void app_main() ;

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "sdkconfig.h"

#define LED_PIN GPIO_NUM_2 // LED en el pin 2 (en muchas placas es el LED
integrado)
#define PINOFF GPIO_NUM_4 // Pin que indica cuando el sistema debe apagarse

void app_main(void)
{
    // Configurar pin como salida
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
    gpio_set_direction(PINOFF, GPIO_MODE_OUTPUT);
    gpio_set_level(PINOFF, 1); // Definir en 1 el pin 4
    uint32_t contador = 0;

    while (1) {
        gpio_set_level(LED_PIN, 1); // Encender LED pin 2
        vTaskDelay(1000 / portTICK_PERIOD_MS); // 1 segundo
        gpio_set_level(LED_PIN, 0); // Apagar LED pin 2
        vTaskDelay(1000 / portTICK_PERIOD_MS); // 1 segundo

        //Hasta aqui hay 2 segundos, cuando pasa al if, el contador aumenta en uno
        //El contador debe aumentar en 5 (5 veces que se repite el ciclo)
        //Es decir cuando hayan pasado 10 segundos del parpadeo del led anterior

        if(contador++==5){
            gpio_set_level(PINOFF, 0); // Definir en 0 el pin 4 APAGA SISTEMA
        }
    }
}
```

Esta actividad nos demuestra un uso útil del MOSFET canal P, ya que, es el principal componente encargado de conectar o desconectar la alimentación de 5 V hacia el VIN del ESP32. Mientras que el controla su activación es el transistor NPN.

También se encuentran componentes con funcionamientos esenciales, como por ejemplo la resistencia de 10K conectada al Source del MOSFET, que funciona como resistencia pull-up, manteniendo esta compuerta en nivel alto por defecto. Mientras que la otra resistencia de 10K en nodo con la base del transistor NPN, funciona como resistencia pull-down, que asegura que la base se encuentre en 0V cuando la ESP32 no conduce.

Capítulo 7

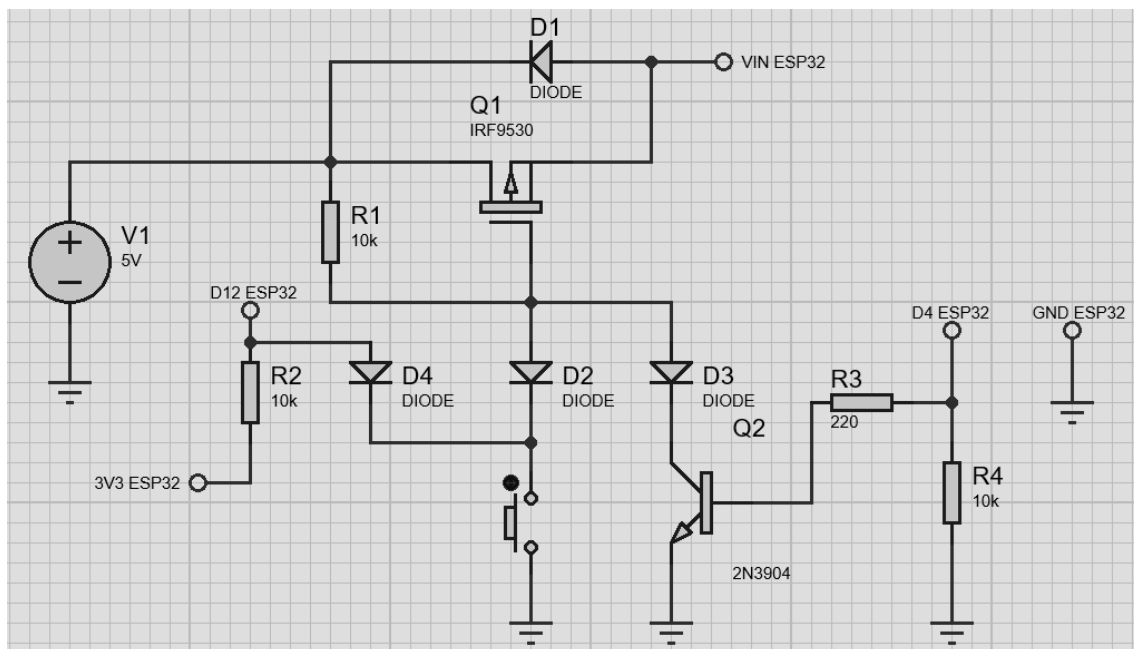
Actividad en clase 3:

En la actividad realizada en el capítulo anterior, se realizó una nueva propuesta, realizar un circuito y un código, que permitan que el sistema se apague, cuando se presione durante 2 segundos un pulsador.

Para esta propuesta se usaron los materiales a continuación:

- Fuente externa de 5V.
- 3 x Resistencias 10 KOhm.
- 1 x Resistencia 220 Ohm.
- Pulsador.
- Transistor NPN 2N3904.
- 4 x Diodo 1N4007.
- MOSFET CANAL P IRF9530.
- Módulo ESP32.

El esquemático propuesto fue el siguiente.



En este caso, el circuito contiene un conjunto de tres sensores que permiten controlar varias señales (D12 de la ESP32, el pulsador y el transistor), forzando al MOSFET a GND sin que se retroalimenten entre sí. Este permite realizar un mejor control por software mediante D12 y D4 del ESP32, sin que se vea afectado el módulo.

El código que se programó fue "Encendido y apagado port 2 seg con pulsador.tx" el cual se encuentra en el directorio del repositorio tambien.

[Encendido y apagado port 2 seg con pulsador.txt](#)

```
extern "C" void app_main() ;

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "sdkconfig.h"

#define LED_PIN GPIO_NUM_2 // LED en el pin 2 (en muchas placas es el LED
integrado)
#define PINOFF GPIO_NUM_4 // Pin que indica cuando el sistema debe apagarse
#define BUTTONOFF GPIO_NUM_12 //Botón de apagado

void app_main(void)
{
    // Configurar pin como salida
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
    gpio_set_direction(PINOFF, GPIO_MODE_OUTPUT);
    gpio_set_level(PINOFF, 1); // Definir en 1 el pin 4
    gpio_set_direction(BUTTONOFF, GPIO_MODE_INPUT);
    gpio_set_pull_mode(BUTTONOFF, GPIO_PULLUP_ONLY); // Pulsador a GND
    //uint32_t contador = 0;

    while (1){

        // Leer estado del botón
        int btnState = gpio_get_level(BUTTONOFF);

        if (btnState == 0) { // Pulsador presionado
            int tiempo = 0;
            while (gpio_get_level(BUTTONOFF) == 0) {
                vTaskDelay(100 / portTICK_PERIOD_MS);
                tiempo += 100;
            }
        }
    }
}
```

```
    if (tiempo >= 1000) { // 2 segundos presionado
        gpio_set_level(PINOFF, 0); // Apagar sistema
        gpio_set_level(LED_PIN, 0);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        esp_restart(); // Reinicia ESP32 al apagarse
    }
}

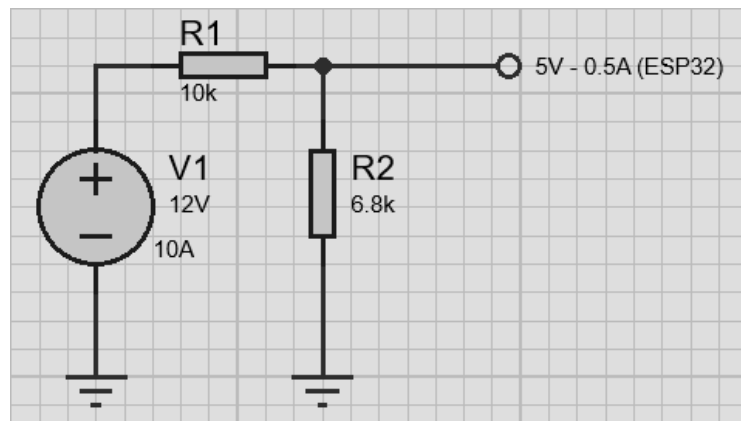
// Si fue un toque corto (<1s) → encender LED
if (tiempo < 1000) {
    gpio_set_level(LED_PIN, 1);
}
}

// vTaskDelay(50 / portTICK_PERIOD_MS);
}
}
```

Capítulo 8

Se propuso un problema a resolver en clase, y es el realizar un circuito que permita obtener 5V de una fuente 12V sin usar algún regulador de voltaje.

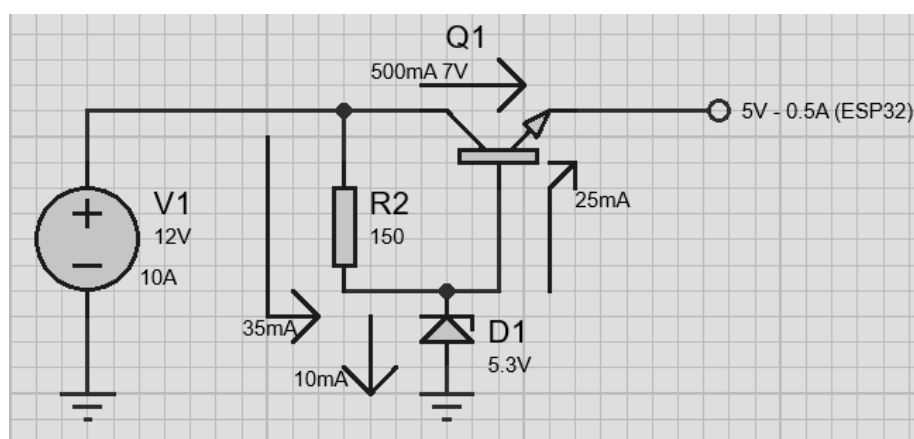
Se explicó por qué no se puede usar un divisor de voltaje, ya que, la potencia puede disminuir para la carga.



$$V = I * R \rightarrow 4.85 = I * 6800 \rightarrow I_1 = \frac{4.85}{6800} = 713\mu A$$

$$\rightarrow I_2 = \frac{12 - 4.85}{10000} = 715\mu A$$

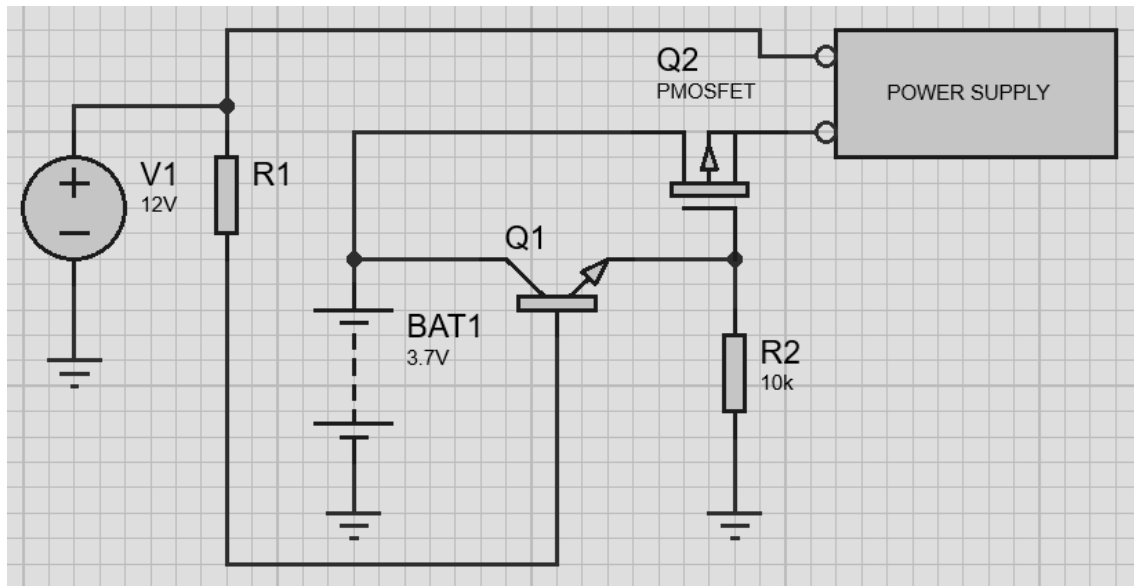
La forma preferida de cumplir con ese objetivo es hacer uso de la combinación entre un transistor NPN, una resistencia 150 Ohm y un diodo Zener de 5V; se usa el transistor para evitar que la impedancia disminuya la potencia. También se puede usar un amplificador operacional para comparar cada una de las señales.



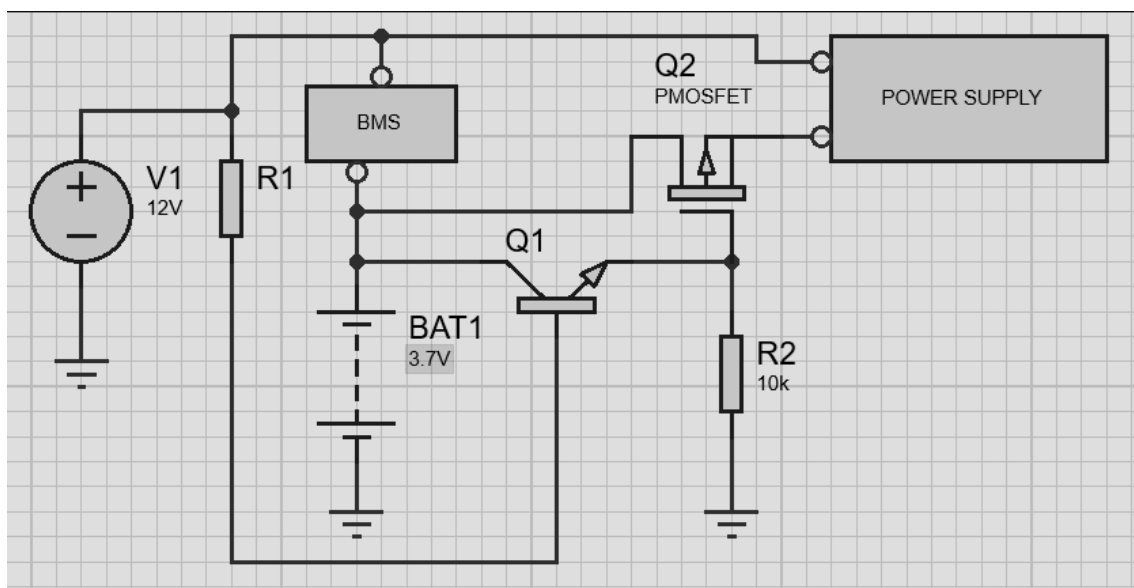
Por último, se explicó una de las razones del uso del condensador en un sistema de carga, ya que, por este mismo componente se generan picos de voltaje constantes como se muestra en la gráfica, cuando el power supply del sistema llega a la máxima carga, esto no afecta al sistema en sí, pero si es algo para tener en cuenta.

Capítulo 9

En esta clase explicó la importancia de usar el BMS (Sistema de Gestión de baterías), el cual, es un sistema que permite gestionar la carga de una batería recargable de litio. Sin embargo, introducimos otro uso del MOSFET canal P, y es el de permitir el switcheo entre alimentación externa y batería, permitiendo elegir qué tipo de fuente se va a usar para alimentar al sistema automáticamente. Esto sin necesidad de realizar cortes o el usar relés. El siguiente esquemático muestra lo anteriormente mencionado.



Adicionalmente, es preferible incluir en el esquemático del switcheo un circuito BMS, realizando mejor las funcionalidades de cada uno de los circuitos para el cuidado de la batería y del sistema.



Recordemos:

- **El BMS** protege y gestiona la batería.
- **El switcheo con MOSFET** asegura una transición limpia entre batería y fuente externa.

Los circuitos mencionados en este y en los anteriores capítulos (incluyendo las actividades en clase con los pulsadores), son esenciales al momento de diseñar el proyecto de la caneca inteligente, no solamente porque es un requisito, sino también es una forma de protección y alimentación necesaria para el sistema del CPU de la ESP32 que vamos a utilizar.

Capítulo 10

Se comenzó a realizar el diseño de la PCB del circuito de la caneca inteligente, con las siguientes recomendaciones:

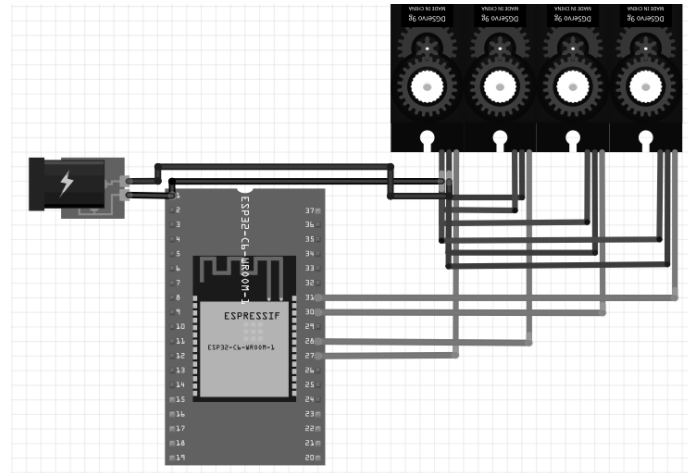
- Debajo de la antena de la ESP32 debe tener un espacio en la PCB sin tierras.
- Si se conecta un chip o un módulo, el pin VCC debe tener si o si un capacitor de 100nF o de 0.1uF.
- Mirar en los datasheet de cada chip que tipo de ruteo se recomienda en las PCB.
- PCB siempre es de 1,6 mm para cuando se diseñe en 3D (esto al momento de realizar una caja protectora para la PCB).
- Diseñar con 4 capas la PCB separadas de la siguiente manera:
 1. Señales.
 2. GND. (Siendo un plano completo o una placa de cobre).
 3. VCC.
 4. Señales.
- Separar ruidos con pistas truehall conectadas a tierra.
- Añadir el uso de un programador para el chip de la ESP32, esto teniendo en cuenta las especificaciones de los pines de este.
- Usar diodos Schottky cerca de los reguladores de voltaje al momento de realizar las fuentes DC-DC, esto para disminuir ruido.

También se realizó una tabla teniendo en cuenta las especificaciones electrónicas de cada componente a usar, como los voltajes, corrientes y potencias.

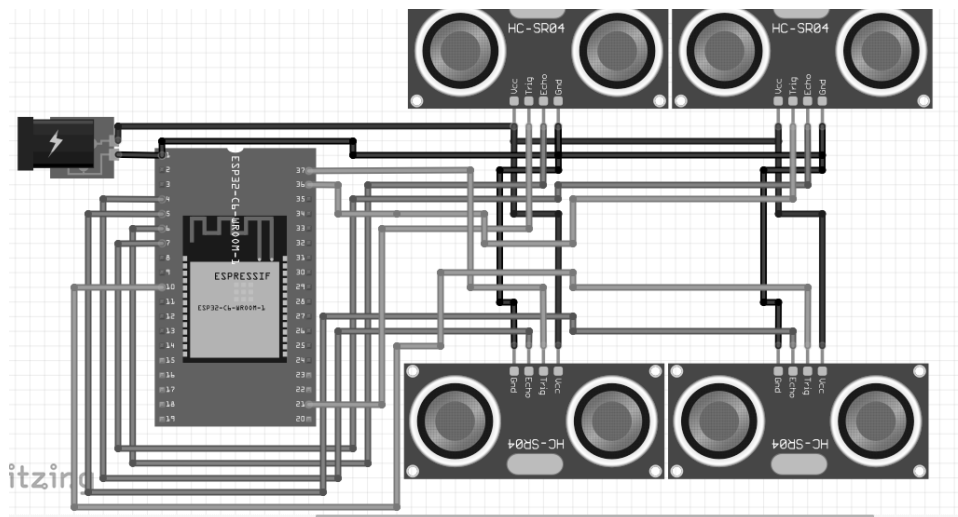
1	Componente	Voltaje (V)	Corriente (mA)	Potencia (W)	
2	Servomotor SG90	4,8 - 6	200	1,2	Se multiplicó 6 * 0,2
3	Servomotor MG995	4,8 - 7,2	500 - 900, máx. 2,5A	2,4	Se multiplicó 4 * 0,5
4	Ultrasonido HC-SR04	5	15	0,075	Se multiplicó 5 * 0,015
5	LED Rojo	1,6 - 2,2	15	0,024	Se multiplicó 1,6 * 0,015
6	LED Blanco Superbrillante	3	15	0,045	Se multiplicó 5 * 0,015
7	Matriz MAX7219 (3xMatriz 1088AS)	4,7 - 5,3	320	1,504	Se multiplicó 4,7 * 0,32
8	ESPCAM	3 - 5	1000	5	Se multiplicó 5 * 1000
9	ESP32	3,3	80	0,264	Se multiplicó 3,3 * 0,08
10	DRV8825	8,2 - 60	1500 (un canal) - 800/canal (4 canales activos)	12,3	Se multiplicó 8,2 * 1,5
11	LM2596	4,5 - 40	Iout = 3A máx.		
12	LM1117	máx. 20	Iout = 800mA máx.		
13	BMS 3S 10A 18650		10 - 13 A máx.		
14	Motor NEMA 17	12 - 48	1 A máx.		

Para tener en cuenta las conexiones de cada periférico en la ESP32, se plantearon las conexiones como se pueden visualizar en las siguientes imágenes, esto gracias al software Fritzing.

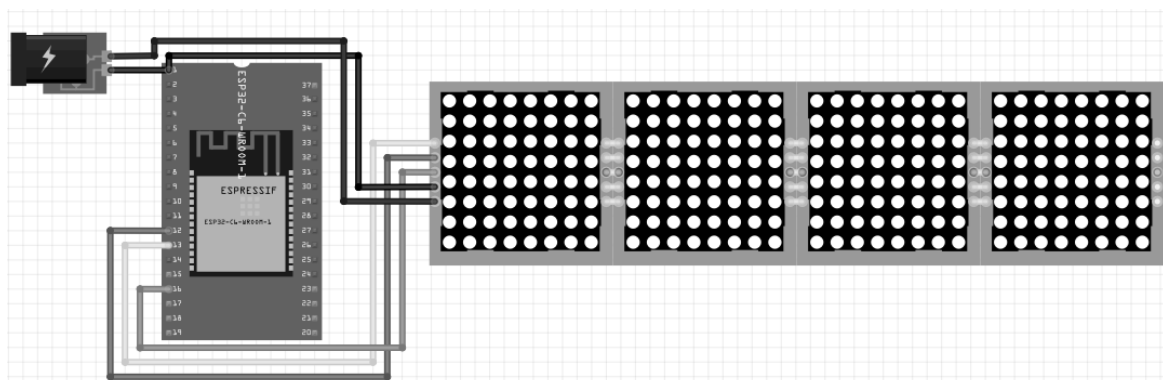
- 4 x Servomotores a ESP32-WROOM-32



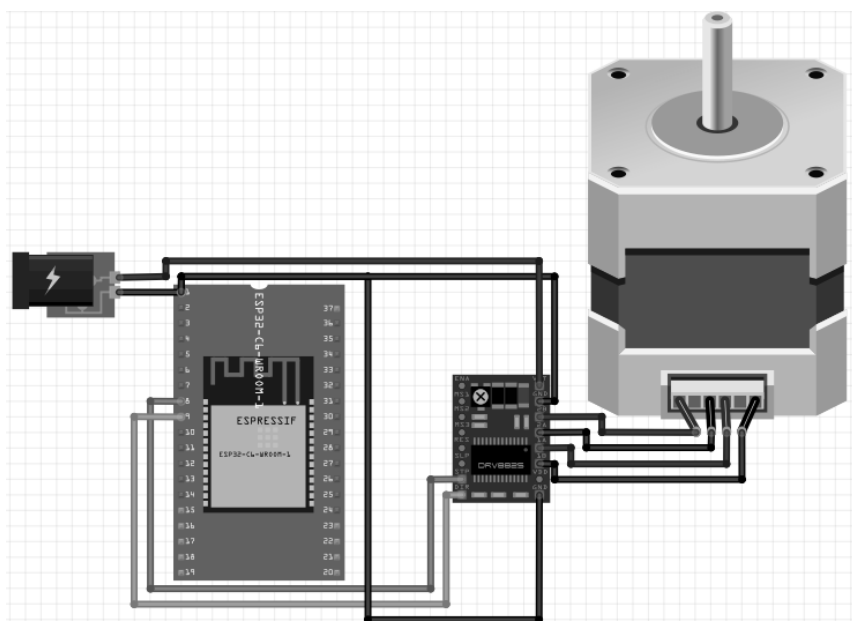
- 4 x Módulos de Ultrasonido a ESP32-WROOM-32



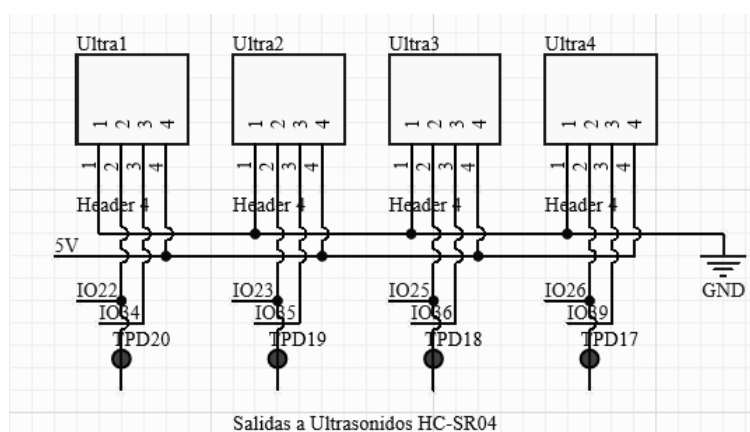
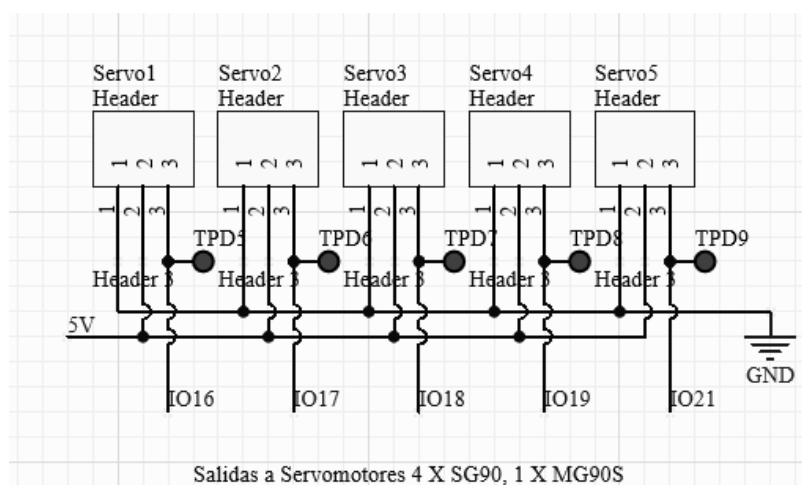
- Pantalla Matriz de LED a ESP32-WROOM-32

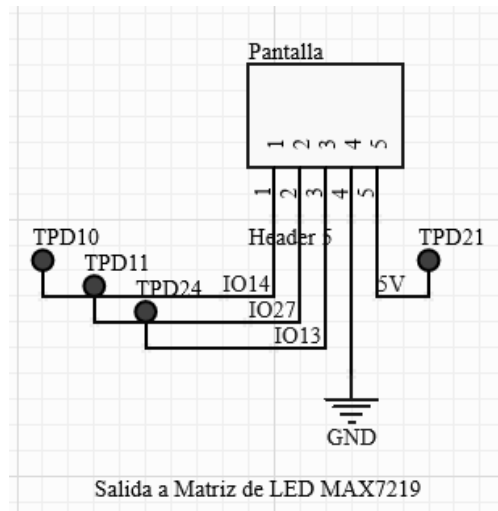


- DRV8825 y Motor Paso a Paso a ESP32-WROOM-32

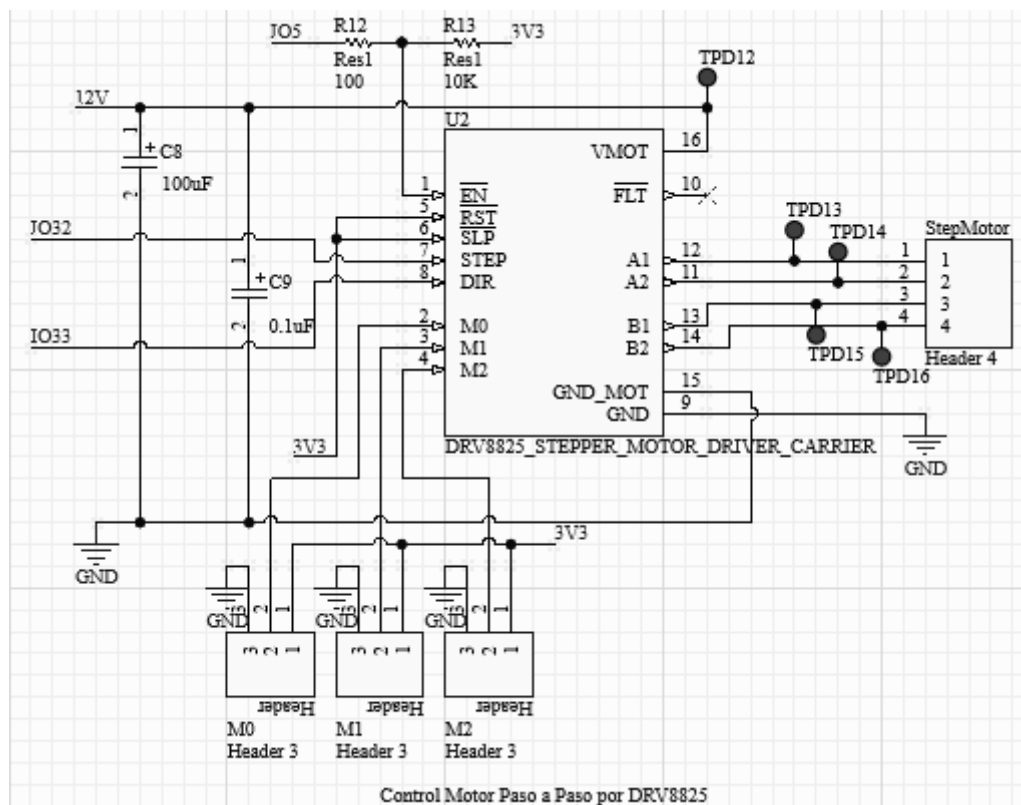


Sus esquemáticos plateados en Altium fueron los siguientes, donde las salidas a cada componente se dan por medio de Headers.

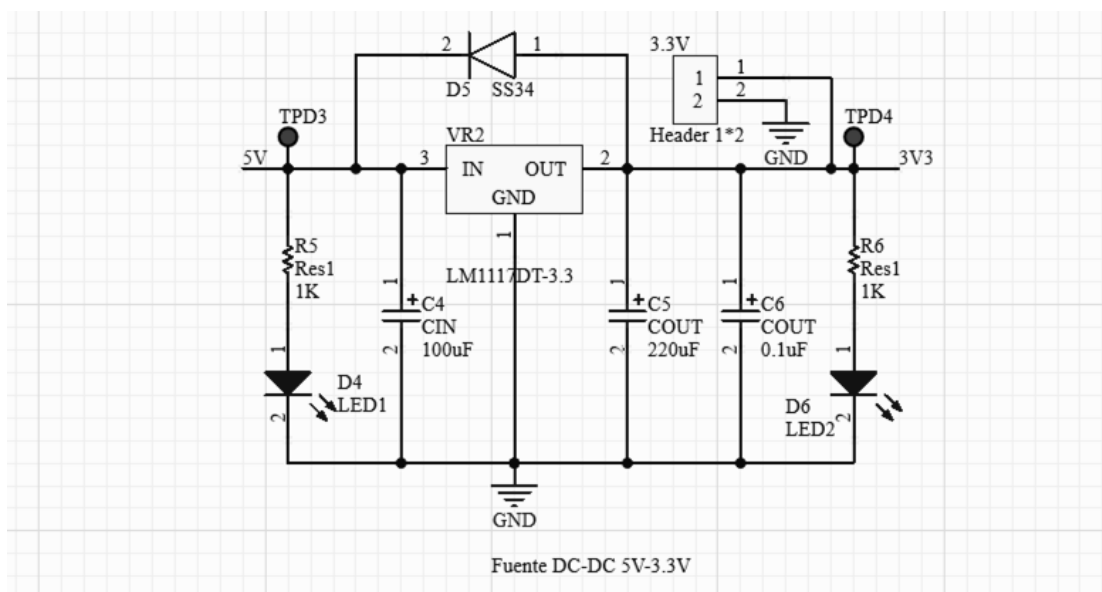
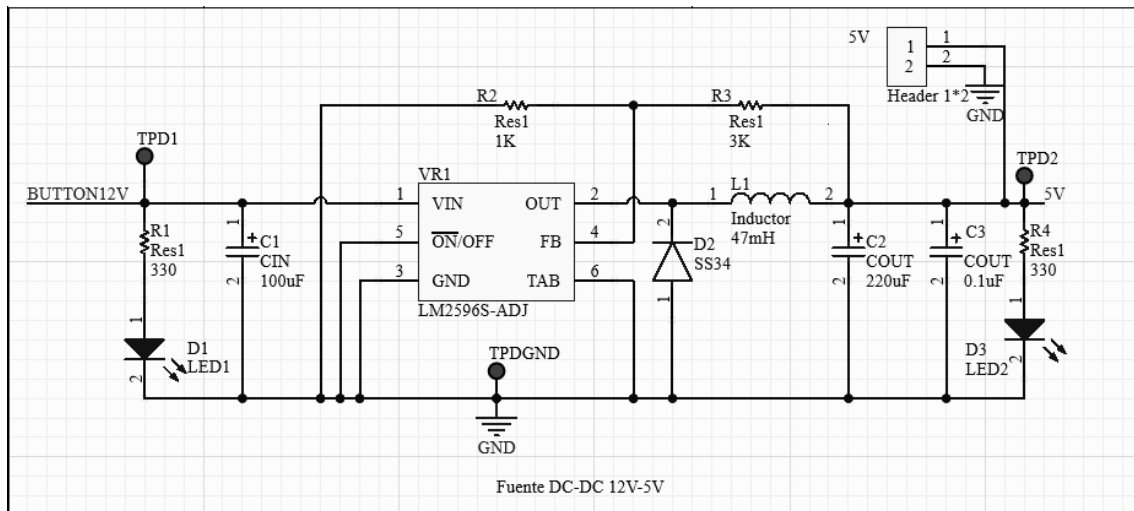




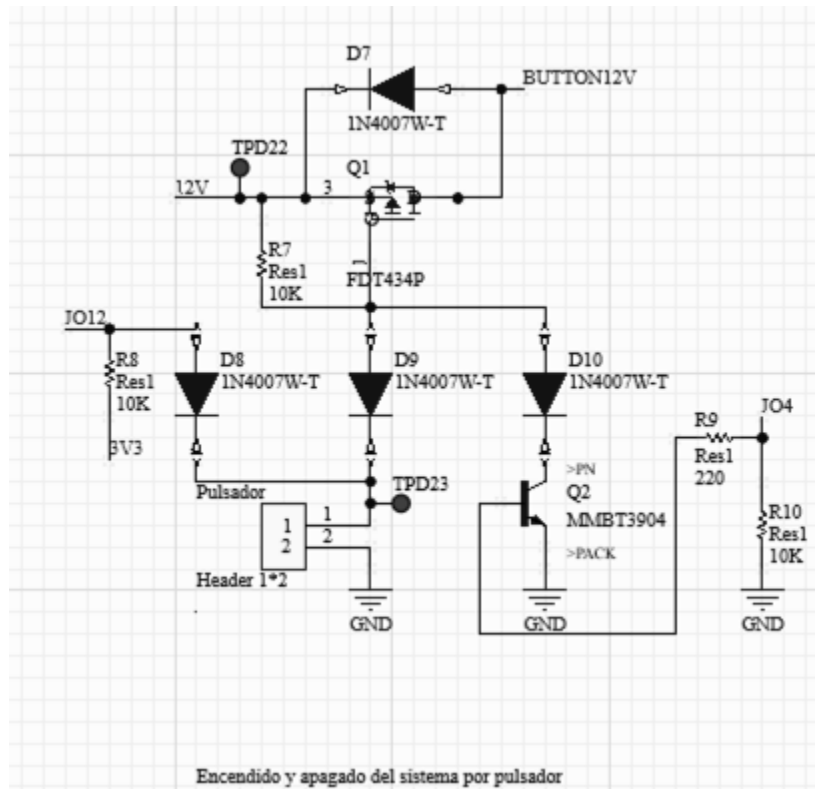
Para el driver DRV8825 para el manejo del motor paso a paso, se tuvo en cuenta configuraciones como el de la posibilidad de control de los micropasos (M0, M1, M2) esto por medio de headers conectados a VCC y a GND, también en el uso de condensadores entre pines, a recomendación del fabricante, y resistencias pull-up en el pin EN para su activación.



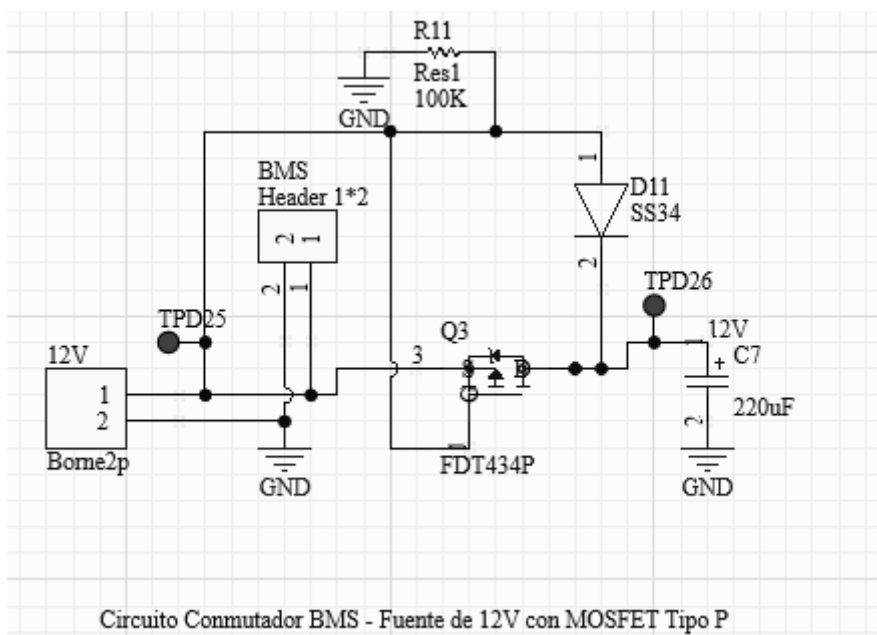
Las fuentes que se van a usar son de 3.3 V y de 5 V, los cuales se van a obtener de una fuente externa de 12 V en un terminal block, estos voltajes alimentaran el resto del sistema.



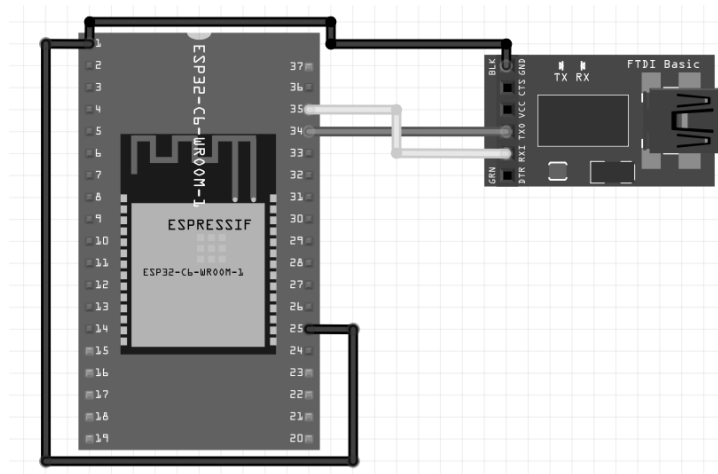
También agregamos el esquemático del circuito correspondiente a la **Actividad 3**, el cual, usaremos para el encendido y apagado del sistema, este encenderá la fuente de 5v y así encender el de 3v, esto nos permitirá encender el procesador ESP, para luego, desde este componente, haremos el respectivo control y programa para hacer que el pulsador sea el mecanismo principal de activación del sistema. En la siguiente captura, el pulsador se mostrará como un header de 2 pines, por lo que nos permitirá conectar el botón de forma externa a la PCB del circuito.



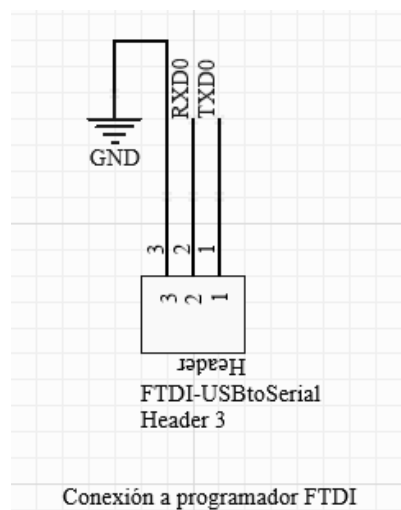
De igual manera, agregaremos un circuito que nos permita switchear automáticamente entre fuente de alimentación AC-DC (12V), y una batería de LITIO 18650 de 3 Celdas (11,1V), denominado **Power Path Management**, además, para protección y control de carga en las baterías, usaremos un BMS (Sistema de gestión de baterías), el circuito será el que se muestra a continuación.



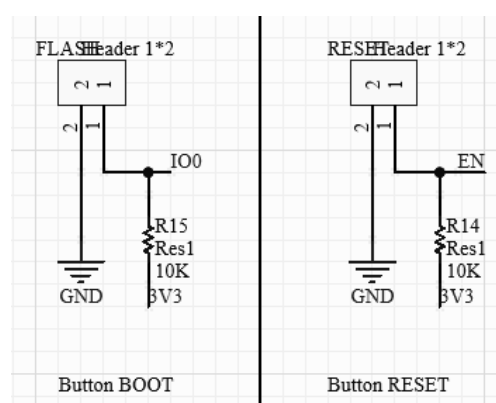
También no debemos de olvidar en señalar los pines que nos permitirán programar la ESP (GPIO 1 y 3), correspondientes a la comunicación TXD0 y RXD0, estos con un pin a GND, conectarlos a un header x3, para que sea posible conectar la PCB a un programador USB to Serial FTDI de forma externa, de la siguiente manera:



Esto se verá en el circuito así:



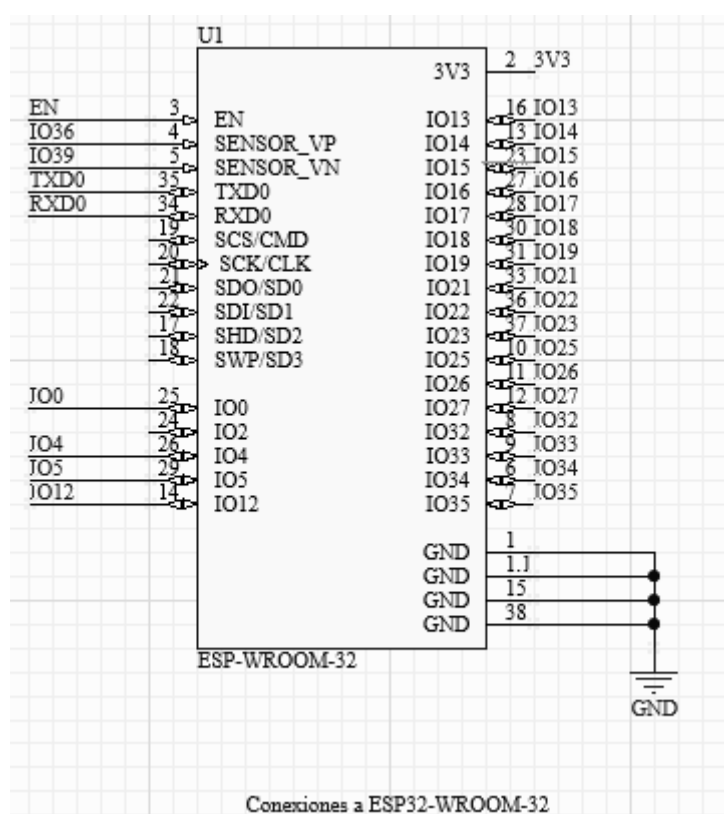
Con ello, las respectivas configuraciones en Pull-up de los pulsadores que harán las funciones esenciales de BOOT y RESET de la ESP32, necesarias al momento de programar.



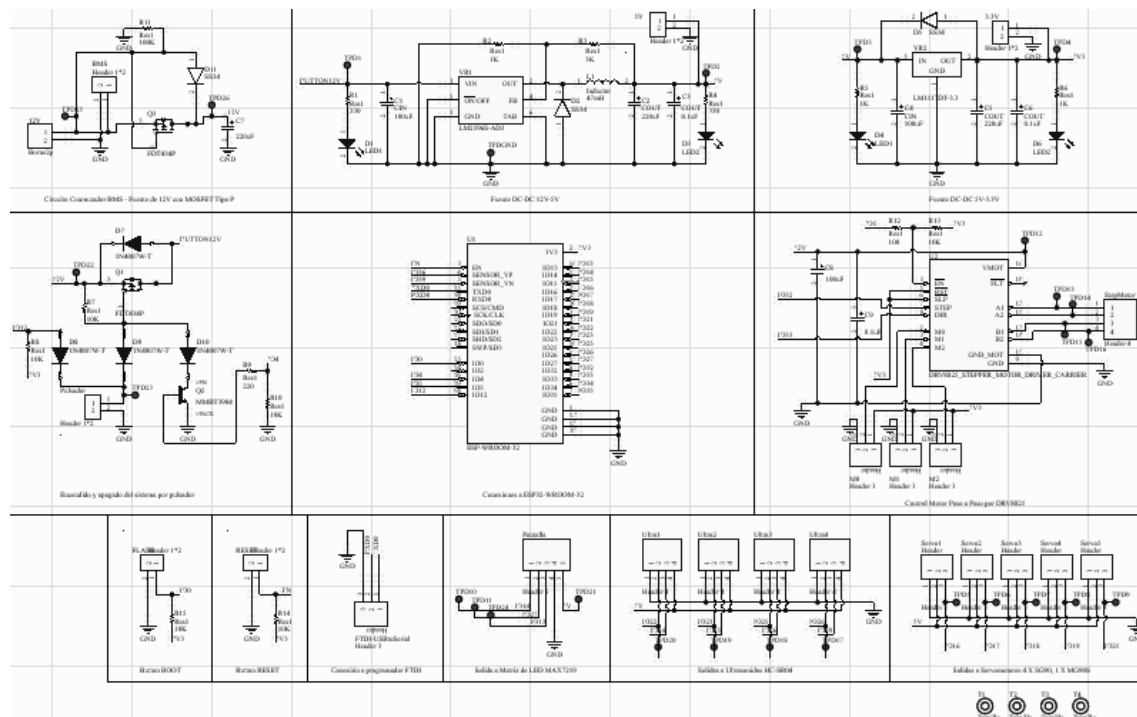
Y agregamos los tornillos a instalar en la PCB.



Por último, las conexiones al chip de la ESP32-WROOM-32, por medio de la herramienta Net.

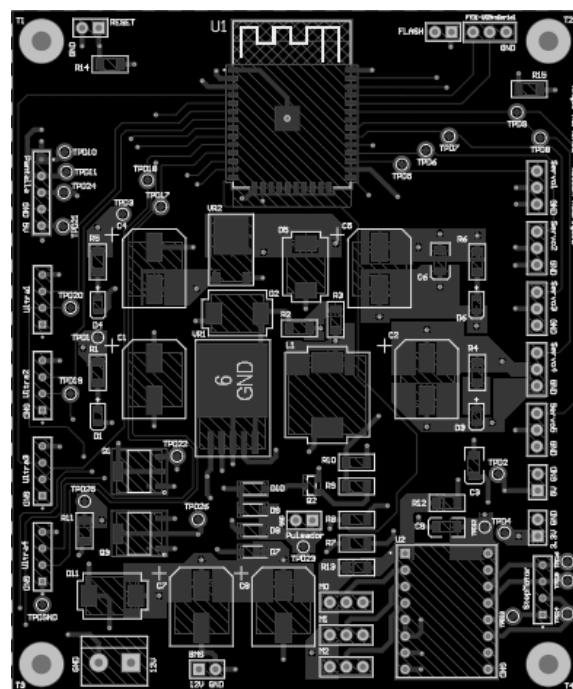


Por lo cual el circuito completo sería el siguiente.



Realización de PCB:

Como se mencionó al inicio del capítulo, la PCB será conformada de 4 capas (Signal-GND-VCC-Signal), siendo la primera y la última capa, las encargadas de distribuir las señales, esto de la siguiente manera:



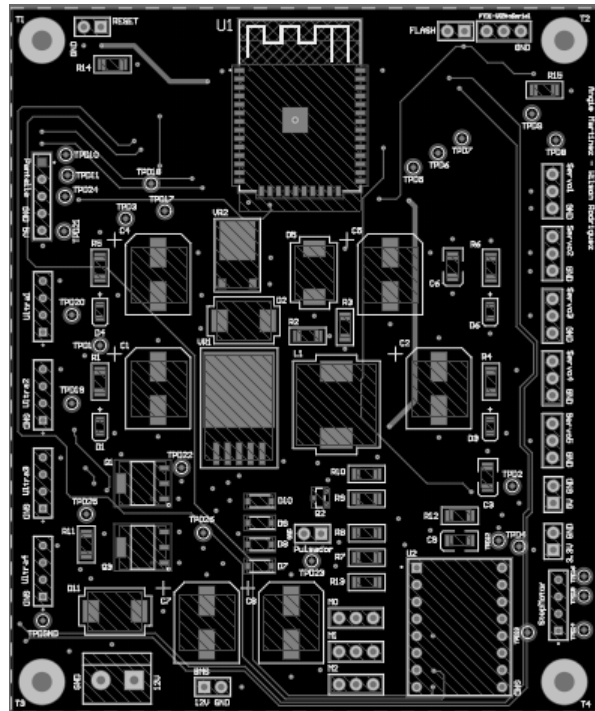


Imagen cuarta capa de señales.

La segunda capa denominada GND, será un plano completo, es decir, una placa de cobre, el cual, solo se conectarán tierras, esta configuración ayudará al circuito a aislar ruido electromagnético y obtener mejores señales.

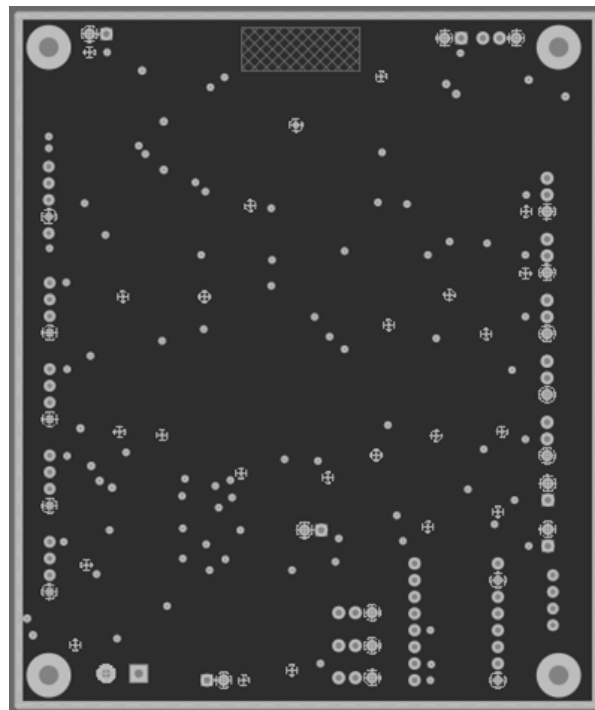


Imagen segunda capa de GND.

Por último, en la tercera capa se van a encontrar las conexiones a VCC, donde se reúnen las fuentes de 3.3V, 5V y 12V, además de conformar las alimentaciones de cada componente.

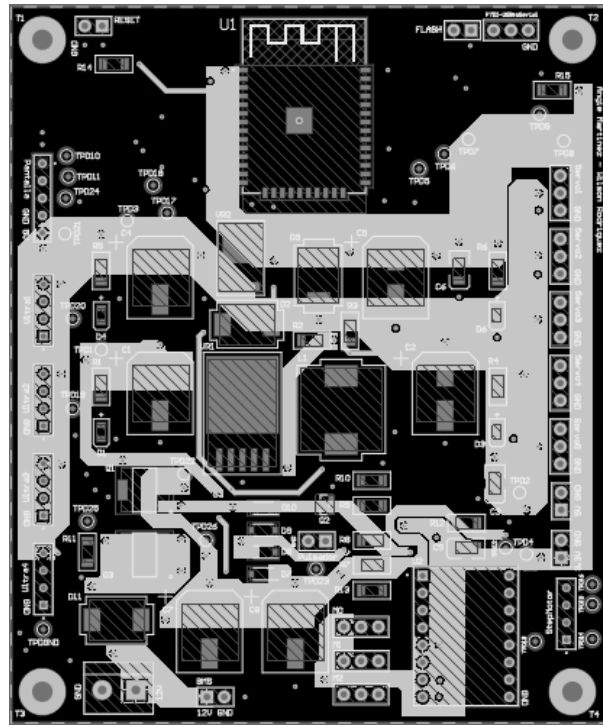


Imagen tercera capa VCC.

Por lo que el circuito completo en PCB sería el siguiente.

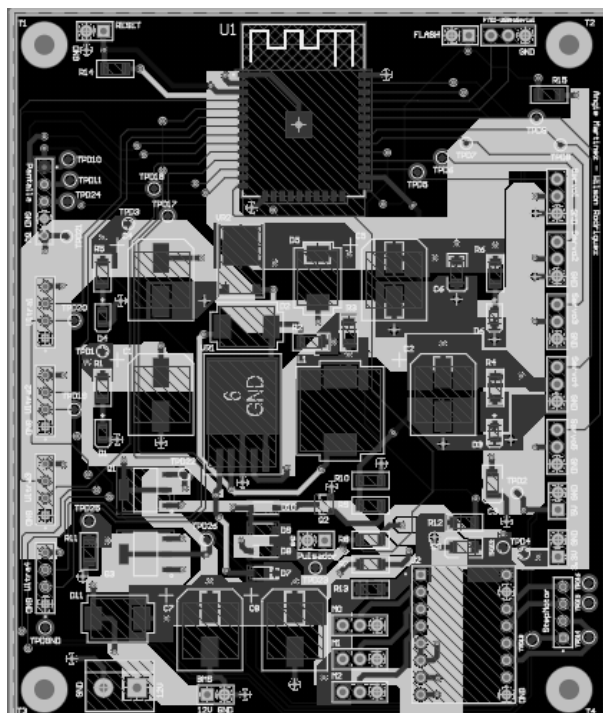
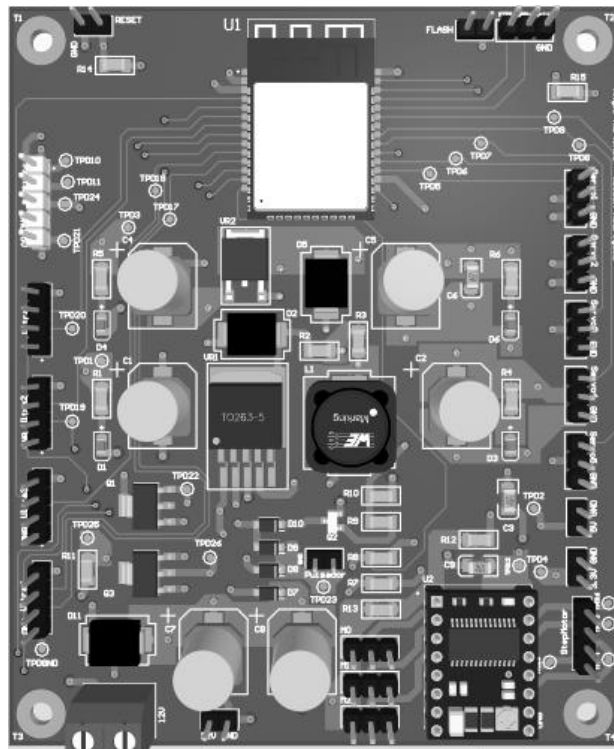
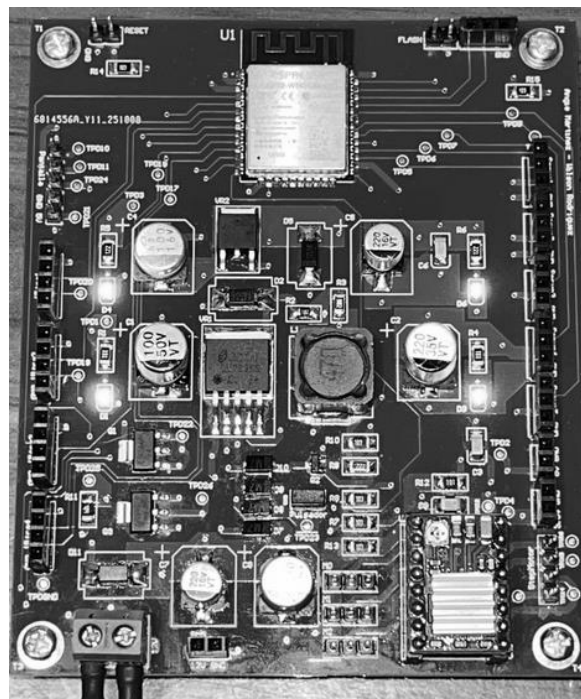


Imagen con todas las capas unidas.

Su figura en 3D con los modelos de los componentes es el siguiente.



El PCB impreso, en funcionamiento y debidamente soldados sus componentes, resultó de la siguiente manera, donde sus salidas de voltajes funcionan correctamente.



Capítulo 11

Para comenzar a programar el Backend de nuestro proyecto, tenemos que familiarizarnos con el uso de servidores y sus componentes. Para ello debemos de entender los fundamentos de los protocolos de red, el modelo de referencia OSI, y el funcionamiento de plataformas de comunicación orientadas a redes locales, servicios de internet y dispositivos IoT. Esto lo profundizamos con el uso de herramientas utilizadas en entornos de desarrollo y administración de redes.

- **Modelo OSI y Protocolos IP**

El modelo OSI (Open Systems Interconnection) es una referencia conceptual que divide la comunicación en redes en siete capas:

1. **Física:** Transmisión de bits a través del medio físico.
2. **Enlace de datos:** Control de acceso al medio y detección de errores.
3. **Red:** Encaminamiento y direccionamiento lógico (IP).
4. **Transporte:** Control de flujo y fiabilidad (TCP/UDP).
5. **Sesión:** Gestión de sesiones entre aplicaciones.
6. **Presentación:** Formato de datos y cifrado.
7. **Aplicación:** Interfaz directa con el usuario (HTTP, SMTP, etc.).

El protocolo IP (Internet Protocol) opera en la capa de red y proporciona una dirección lógica de 4 bytes que permite identificar dispositivos en una red. A diferencia de la dirección MAC, que es física y única para cada interfaz de red, la IP puede ser asignada dinámicamente mediante servicios como DHCP, comúnmente configurado en el Firewall. [16]

- **Direccionamiento y Servicios de Red**

Dirección MAC: Identificador físico de la interfaz de red. Aunque el usuario no puede modificarla directamente, el sistema puede hacerlo en contextos de ciberseguridad.

Interfaces de conexión: Cada interfaz (Wi-Fi, Ethernet, etc.) muestra una dirección MAC distinta.

Gateway: Puerta de enlace entre redes LAN (Local Area Network) y WAN (Wide Area Network).

ISP (Internet Service Provider): Proveedor de acceso a internet, conectado directamente al firewall.

DNS (Domain Name System): Servicio que traduce nombres de dominio a direcciones IP.

Protocolos adicionales: ICMP (diagnóstico de red), SMTP (correo electrónico).

- **Protocolos de Transporte y Comunicación**

TCP (Transmission Control Protocol): Orientado a conexión. Garantiza la entrega de datos mediante confirmaciones. Ideal para transacciones críticas.

UDP (User Datagram Protocol): No orientado a conexión. Se utiliza cuando la velocidad es prioritaria sobre la fiabilidad.

MQTT (Message Queuing Telemetry Transport): Protocolo ligero diseñado para IoT, con paquetes pequeños y bajo consumo de recursos.

Overhead: Información adicional en los paquetes que no aporta valor directo, pero es necesaria para el control de la comunicación.

- **Herramientas y Sistemas**

OpenWrt: Sistema operativo de código abierto para dispositivos de red, como routers.

Portal cautivo: Red abierta que redirige al usuario a una página web para autenticación mediante datos personales.

Programa Hercules: Herramienta de simulación de comunicación por protocolo HTTP. Ejemplo: conexión a 127.0.0.1:3000.

Postman: Plataforma para pruebas y desarrollo de APIs. Permite enviar solicitudes HTTP, visualizar respuestas, y automatizar flujos de prueba en entornos de red y backend.

MongoDB: Base de datos NoSQL orientada a documentos, ideal para aplicaciones que requieren flexibilidad en el almacenamiento de datos. Se integra fácilmente con plataformas web, IoT y sistemas distribuidos.

- **Plataformas y Bases de Datos**

En entornos educativos o administrativos, las plataformas permiten la integración de múltiples dispositivos y usuarios. Por ejemplo, una plataforma de notas conecta al servidor con los usuarios finales, quienes acceden a la información mediante interfaces web.

Para el diseño de bases de datos, se recomienda el uso de herramientas como **diagram.io**, que permiten la creación de tablas y relaciones de manera visual y estructurada.

- **Actividad en clase 4:**

Se realizó un pequeño ejemplo para establecer una conexión TCP desde un ESP32 hacia un servidor remoto a través de WiFi. Solamente para visualizar el envío y recepción de mensajes, esto también se visualizó con el programa **Hercules**, y se programó en la interfaz de **Visual Studio Code**. Comenzaremos con las partes esenciales en la programación de este tipo de tareas.

Primero se debe de nombrar el cliente TCP en ESP32.

```
extern "C" void app_main();
```

Segundo hay que añadir librerías.

```
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "lwip/sockets.h"
```

Tercero configuramos la red.

```
#define WIFI_SSID "ROBOT1"
#define WIFI_PASS "Holamundo"
#define SERVER_IP "10.0.0.220"
#define SERVER_PORT 5000
```

Para así continuar con las siguientes líneas de código que nos permitirán realizar la conexión entre la ESP32 y el WIFI, esto permitiendo mostrar cada 5 segundos en el servidor maestro la siguiente frase: "Temp: 10 C\n".

```

extern "C" void app_main() ;

#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "lwip/sockets.h"

#define WIFI_SSID      "ROBOT1"
#define WIFI_PASS      "Holamundo"
#define SERVER_IP      "10.0.0.220" // Cambia por la IP de tu PC/servidor
#define SERVER_PORT     5000

static const char *TAG = "TCP_CLIENT";

static void wifi_event_handler(void* arg, esp_event_base_t event_base,
                               int32_t event_id, void* event_data) {
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
        esp_wifi_connect();
        ESP_LOGI(TAG, "Reintentando conexión WiFi...");
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t* event = (ip_event_got_ip_t*) event_data;
        ESP_LOGI(TAG, "Conectado. IP: " IPSTR, IP2STR(&event->ip_info.ip));
    }
}

void wifi_init(void) {
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT,
                                                         ESP_EVENT_ANY_ID,
                                                         &wifi_event_handler,
                                                         NULL,
                                                         NULL));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT,
                                                         IP_EVENT_STA_GOT_IP,
                                                         &wifi_event_handler,
                                                         NULL,
                                                         NULL));

    wifi_config_t wifi_config = {
        .sta = {
            .ssid = WIFI_SSID,
            .password = WIFI_PASS,
        },
    };

    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA) );
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config) );
    ESP_ERROR_CHECK(esp_wifi_start() );
}

```

```

void tcp_client_task(void *pvParameters) {
    char *msg = "Temp: 10 C\n";
    char rx_buffer[128];
    struct sockaddr_in dest_addr;
    dest_addr.sin_addr.s_addr = inet_addr(SERVER_IP);
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(SERVER_PORT);

    while (1) {
        int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);
        if (sock < 0) {
            ESP_LOGE(TAG, "Error creando socket");
            vTaskDelay(2000 / portTICK_PERIOD_MS);
            continue;
        }

        ESP_LOGI(TAG, "Conectando a %s:%d", SERVER_IP, SERVER_PORT);
        int err = connect(sock, (struct sockaddr *)&dest_addr, sizeof(dest_addr));
        if (err != 0) {
            ESP_LOGE(TAG, "Error en connect: errno %d", errno);
            close(sock);
            vTaskDelay(2000 / portTICK_PERIOD_MS);
            continue;
        }

        ESP_LOGI(TAG, "Conectado al servidor");

        while (1) {
            // === Enviar cada 1 segundo ===
            int sent = send(sock, msg, strlen(msg), 0);
            if (sent < 0) {
                ESP_LOGE(TAG, "Error enviando datos: errno %d", errno);
                break; // salimos y reintentamos conexión
            }
            ESP_LOGI(TAG, "Mensaje enviado: %s", msg);

            // === Recibir respuesta (no bloqueante largo) ===
            int len = recv(sock, rx_buffer, sizeof(rx_buffer) - 1, MSG_DONTWAIT);
            if (len > 0) {
                rx_buffer[len] = 0;
                ESP_LOGI(TAG, "Recibido: %s", rx_buffer);
            }

            vTaskDelay(2000 / portTICK_PERIOD_MS); // esperar 1s
        }

        shutdown(sock, 0);
        close(sock);

        // Reintentar conexión en 5 segundos
        vTaskDelay(5000 / portTICK_PERIOD_MS);
    }
}

```

```
void app_main(void) {
    ESP_ERROR_CHECK(nvs_flash_init());
    wifi_init();

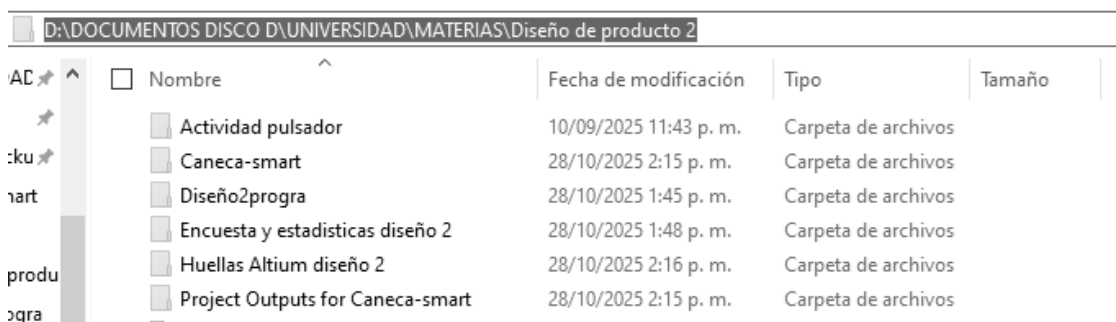
    // Esperar WiFi antes de conectar
    vTaskDelay(5000 / portTICK_PERIOD_MS);

    xTaskCreate(tcp_client_task, "tcp_client", 4096, NULL, 5, NULL);
}
```

- **Actividad en clase 5:**

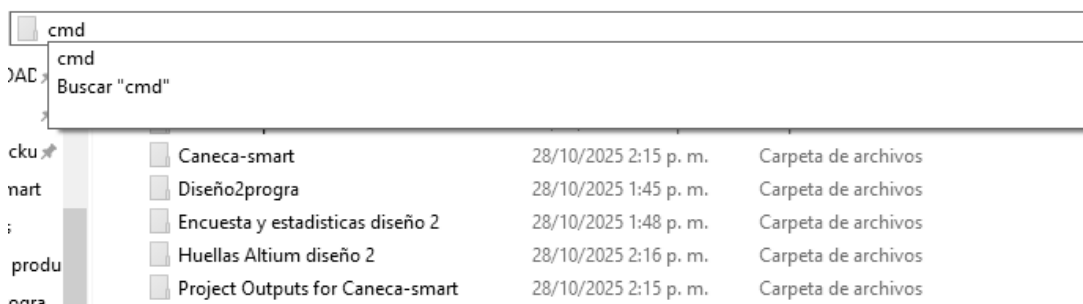
Realizaremos un código de una aplicación backend desarrollada con **Node.js** y el servidor framework **Express**, esto para gestionar datos simulados API REST de canecas (en el caso de nuestro proyecto), ideal para pruebas con Postman o integración con el ESP32.

Para ello comenzaremos creando una carpeta desde un terminal, para ello entramos al archivo en donde queremos crear la carpeta, esto desde nuestro administrador de archivos.



Nombre	Fecha de modificación	Tipo	Tamaño
Actividad pulsador	10/09/2025 11:43 p. m.	Carpeta de archivos	
Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	
Diseño2progra	28/10/2025 1:45 p. m.	Carpeta de archivos	
Encuesta y estadísticas diseño 2	28/10/2025 1:48 p. m.	Carpeta de archivos	
Huellas Altium diseño 2	28/10/2025 2:16 p. m.	Carpeta de archivos	
Project Outputs for Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	

Y en la barra de búsqueda seleccionada, escribiremos “cmd”, para entrar en la terminal desde esta ruta directamente.



Nombre	Fecha de modificación	Tipo	Tamaño
Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	
Diseño2progra	28/10/2025 1:45 p. m.	Carpeta de archivos	
Encuesta y estadísticas diseño 2	28/10/2025 1:48 p. m.	Carpeta de archivos	
Huellas Altium diseño 2	28/10/2025 2:16 p. m.	Carpeta de archivos	
Project Outputs for Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	

Nos aparecerá la ventana del terminal con la ruta seleccionada, procedemos a escribir la instrucción “mkdir [nombre de la carpeta]” para hacer la creación de la carpeta.


```

C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6456]
(c) Microsoft Corporation. Todos los derechos reservados.
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2>mkdir TablaDise2_

```

Entraremos a la carpeta creada desde la terminal usando “cd [nombre de la carpeta]”.

```

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2>cd TablaDise2

```

Ya dentro de la carpeta creada, inicializaremos la creación del paquete “**PACKAGE.JSON**” con la siguiente instrucción “npm init -y”.

```

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>npm init -y
Wrote to D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2\package.json:

{
  "name": "tabladise2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

```

Instalaremos las dependencias necesarias para el funcionamiento de nuestro programa (**node-modules y package-lock-json**). Esto con la instrucción “npm install express cors”.

```

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>npm install express cors
added 70 packages, and audited 71 packages in 7s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

Creamos el archivo principal de **INDEX.JS** donde realizaremos el código del programa. Esto con la instrucción “index.js”.

```

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>echo > index.js

```

Opcionalmente, creamos una carpeta .dist, esto nos permitirá compilar código en otros programas, con “mkdir dist”.

```

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>mkdir dist

```

Ya con lo anterior creado, comenzaremos a realizar un código donde podremos visualizar una tabla en postman **en el puerto 3000**.

```
const express = require('express');
const cors = require('cors');
const app = express();
const PORT = 3000;

app.use(cors());
app.use(express.json());

const canecas = [
  { id: 1, nombre: "Caneca plásticos", botellas: 10, bolsas: 3, otros: 5 },
  { id: 2, nombre: "Caneca papel y cartón", papeles: 7, cartones: 2, otros: 0 },
  { id: 3, nombre: "Caneca metal", latas: 1, otros: 0 },
];

//Ruta raíz
app.get('/', (req, res) => {
  res.send('Servidor funcionando 🚀 – usa /canecas para ver los datos');
});

//Ruta de los datos
app.get('/canecas', (req, res) => {
  res.json(canecas);
});

app.listen(PORT, () => {
  console.log(`Servidor iniciado en http://localhost:${PORT}`);
});
```

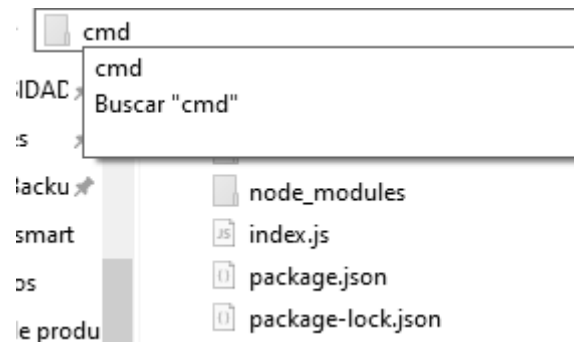
Antes de iniciar Postman, debemos de iniciar a simular el programa que acabamos de escribir, para ello vamos a entrar al cmd del sistema, entrando primero al directorio o a la carpeta donde lo guardamos, en este caso entraremos a la carpeta “TablaDise2”.

Este equipo > Nuevo vol (D:) > DOCUMENTOS DISCO D > UNIVERSIDAD > MATERIAS > Diseño de producto 2				
☐	Nombre	Fecha de modificación	Tipo	Tamaño
☐	Actividad pulsador	10/09/2025 11:43 p. m.	Carpeta de archivos	
☐	Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	
☐	Diseño2progra	28/10/2025 1:45 p. m.	Carpeta de archivos	
☐	Encuesta y estadísticas diseño 2	28/10/2025 1:48 p. m.	Carpeta de archivos	
☐	Huellas Altium diseño 2	28/10/2025 2:16 p. m.	Carpeta de archivos	
☐	Project Outputs for Caneca-smart	28/10/2025 2:15 p. m.	Carpeta de archivos	
☒	TablaDise2	28/10/2025 3:47 p. m.	Carpeta de archivos	

Ya dentro de la carpeta, señalaremos la ruta en la barra de búsqueda.

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2				
	Nombre	Fecha de modificación	Tipo	Tamaño
	dist	28/10/2025 3:47 p. m.	Carpeta de archivos	
	node_modules	28/10/2025 3:43 p. m.	Carpeta de archivos	
	index.js	28/10/2025 3:51 p. m.	Archivo de origen ...	1 KB
	package.json	28/10/2025 3:43 p. m.	Archivo de origen ...	1 KB
	package-lock.json	28/10/2025 3:43 p. m.	Archivo de origen ...	30 KB

Para escribir después el comando “cmd”.



Iniciado el terminal, podemos ver la versión instalada de node con la instrucción “node -v”.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6456]
(c) Microsoft Corporation. Todos los derechos reservados.
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>node -v
v22.20.0
```

Instalaremos el paquete NODEMON con la línea “npm install -g nodemon”.

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>npm install -g nodemon
changed 29 packages in 4s
4 packages are looking for funding
run `npm fund` for details
```

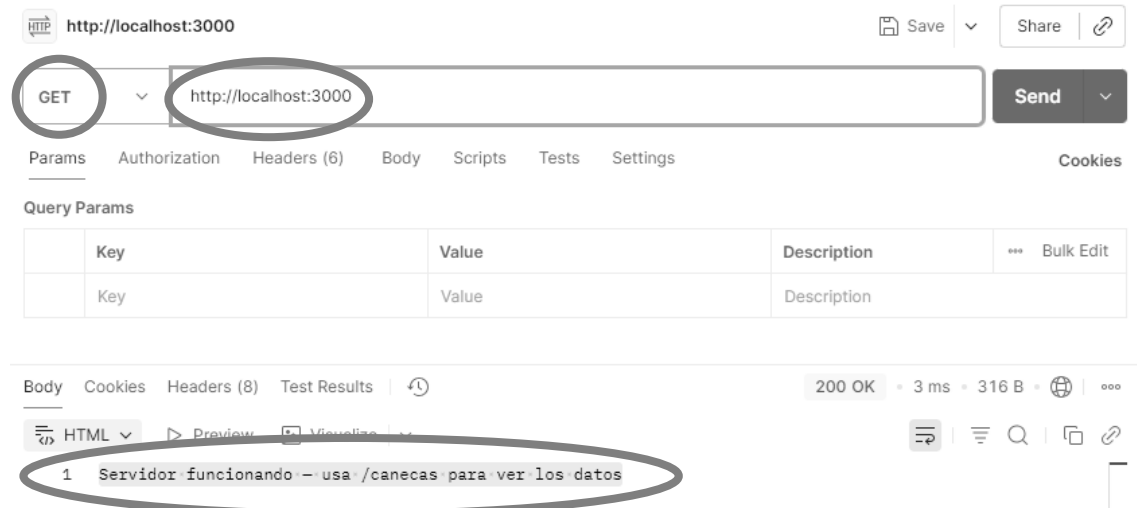
Podemos iniciar el servidor de dos maneras, usando el paquete node, o el paquete nodemon. Cada uno con sus propias instrucciones. En el caso de node, podemos iniciarlo así: “node index.js”.

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>node index.js
Servidor iniciado en http://localhost:3000
```

Y en el caso de nodemon podemos iniciarlo así: “nodemon index.js”

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\TablaDise2>NODEMON INDEX.JS
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: JS,json
[nodemon] starting `node INDEX.JS`
Servidor iniciado en http://localhost:3000
```

Desde que nos aparezca en el terminal “Servidor iniciado en `http://localhost:3000`”, podemos dar inicio al uso del programa **Postman**. Para obtener la información que nos quiere enviar el servidor debemos tener en cuenta que la instrucción debe decir “**GET**” para así luego escribir la dirección del servidor y el puerto que programamos “**`http://localhost:3000`**” para luego enviarlo con “**Send**”. Si se ha realizado correctamente el procedimiento aparecerá el mensaje resultado en el “**Body**” diciendo “**Servidor funcionando – usa /canecas para ver los datos**”.



Lo siguiente que realizaremos dentro del postman es verificar si se pueden visualizar las tablas, para ello mantenemos activa la instrucción “**GET**”, y en la ruta de búsqueda agregaremos “**/canecas**” y luego seleccionaremos la opción de “**Send**”, después de esto, podemos visualizar en el “**Body**” las tablas impresas.

GET: <http://localhost:3000/canecas>

Body:

```
[
{
  "id": 1,
  "nombre": "Caneca plasticos",
  "botellas": 10,
  "bolsas": 3,
  "otros": 5
},
{
  "id": 2,
  "nombre": "Caneca papel y carton",
  "papeles": 7,
  "cartones": 2,
```

```

"otros": 0
},
{
  "id": 3,
  "nombre": "Caneca metal",
  "latas": 1,
  "otros": 0
}
]

```

Las tablas de las canecas se verían de la siguiente manera:

Identificación: 1	Identificación: 2	Identificación: 3
Caneca: plásticos	Caneca: papel y cartón	Caneca: metal
Botellas: 10	Papeles: 7	Latas: 1
Bolsas: 3	Cartones: 2	Otros: 0
Otros: 5	Otros: 0	

- **Actividad en clase 6:**

En el mismo programa de la actividad 5, podemos hacer que desde el postman sea funcional para diferentes actividades con las tablas, como, por ejemplo, el agregar una caneca nueva, editar alguna existente, o incluso eliminar alguna que no necesitemos, para ello realizaremos el siguiente código.

```

const express = require('express');
const cors = require('cors');
const app = express();
const PORT = 3000;

app.use(cors());
app.use(express.json());

// Datos simulados
let canecas = [
  { id: 1, nombre: "Caneca plasticos", botellas: 10, bolsas: 3, otros: 5 },
  { id: 2, nombre: "Caneca papel y carton", papeles: 7, cartones: 2, otros: 0 },
  { id: 3, nombre: "Caneca metal", latas: 1, otros: 0 },
];

// RUTA BASE
app.get('/', (req, res) => {
  res.send('Servidor funcionando – usa /canecas para ver los datos');
});

// GET - Ver todas las canecas
app.get('/canecas', (req, res) => {
  res.json(canecas);
});

```

```
// POST - Agregar una nueva caneca
app.post('/canecas', (req, res) => {
  const nuevaCaneca = req.body;
  nuevaCaneca.id = canecas.length ? canecas[canecas.length - 1].id + 1 : 1;
  canecas.push(nuevaCaneca);
  res.status(201).json({ mensaje: 'Caneca agregada', caneca: nuevaCaneca });
});

// PUT - Editar una caneca existente
app.put('/canecas/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const datosActualizados = req.body;

  const index = canecas.findIndex(c => c.id === id);
  if (index === -1) {
    return res.status(404).json({ mensaje: 'Caneca no encontrada' });
  }

  canecas[index] = { ...canecas[index], ...datosActualizados };
  res.json({ mensaje: 'Caneca actualizada', caneca: canecas[index] });
});

// DELETE - Eliminar una caneca
app.delete('/canecas/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = canecas.findIndex(c => c.id === id);
  if (index === -1) {
    return res.status(404).json({ mensaje: 'Caneca no encontrada' });
  }

  const eliminada = canecas.splice(index, 1);
  res.json({ mensaje: 'Caneca eliminada', caneca: eliminada[0] });
});

// Iniciar servidor
app.listen(PORT, () => {
  console.log(`Servidor iniciado en http://localhost:${PORT}`);
});
```

Desde postman al seleccionar **GET** con el enlace, podremos **ver** las tablas.

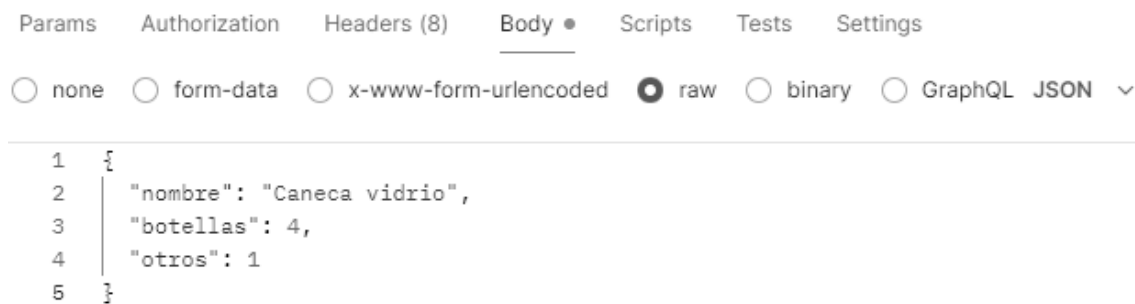
GET http://localhost:3000/canecas

Identificación: 1	Identificación: 2	Identificación: 3
Caneca: plásticos	Caneca: papel y cartón	Caneca: metal
Botellas: 10	Papeles: 7	Latas: 1
Bolsas: 3	Cartones: 2	Otros: 0
Otros: 5	Otros: 0	

Con la instrucción **POST** podremos **agregar** una nueva caneca con el mismo enlace, sin embargo, en este caso debemos agregar la información nueva que

queramos añadir, para ello nos ubicamos en **Body** y seleccionamos **raw** para crear el nuevo código.

POST http://localhost:3000/canecas



En postman se mostraría de la siguiente manera si lo enviamos como se ve en la imagen anterior.

```

{
  "mensaje": "Caneca agregada",
  "caneca": {
    "nombre": "Caneca vidrio",
    "botellas": 4,
    "otros": 1,
    "id": 4
  }
}

```

Si verificamos con **GET** para observar de nuevo todas las tablas, estarían de la siguiente manera.

Identificación: 1	Identificación: 2	Identificación: 3	Identificación: 4
Caneca: plásticos	Caneca: papel y cartón	Caneca: metal	Caneca: vidrio
Botellas: 10	Papeles: 7	Latas: 1	Botellas: 4
Bolsas: 3	Cartones: 2	Otros: 0	Otros: 1
Otros: 5	Otros: 0		

Para **editar** alguna caneca existente en nuestra base de datos, hay que colocar la instrucción **PUT**, sin embargo, en el enlace hay que especificar que caneca queremos editar, esto indicando con el número de la identificación de la caneca al lado de un “/”, de la siguiente manera (si por ejemplo vamos a editar la caneca 2, o la caneca de papel y cartón).

PUT http://localhost:3000/canecas/2

Y en el **Body** del programa en **raw** añadiremos lo que queremos cambiar, como por ejemplo cambiar la cantidad de “papeles” y “otros”.

```
{
  "papeles": 10,
  "otros": 2
}
```

Como resultado se mostrará de la siguiente manera.

```
{
  "mensaje": "Caneca actualizada",
  "caneca": {
    "id": 2,
    "nombre": "Caneca papel y carton",
    "papeles": 10,
    "cartones": 2,
    "otros": 2
  }
}
```

Y si miramos con **GET** las tablas serían así (podemos observar que los “papeles” no son 7 y los “otros” no son 0):

Identificación: 1	Identificación: 2	Identificación: 3	Identificación: 4
Caneca: plásticos	Caneca: papel y cartón	Caneca: metal	Caneca: vidrio
Botellas: 10	Papeles: 10	Latas: 1	Botellas: 4
Bolsas: 3	Cartones: 2	Otros: 0	Otros: 1
Otros: 5	Otros: 2		

Para **eliminar** una caneca, solamente tenemos que indicar su número de identificación al lado del enlace, con la instrucción de **DELETE**. Si por ejemplo queremos eliminar la cuarta caneca, el link y su resultado en postman se verían de la siguiente manera.

DELETE http://localhost:3000/canecas/4

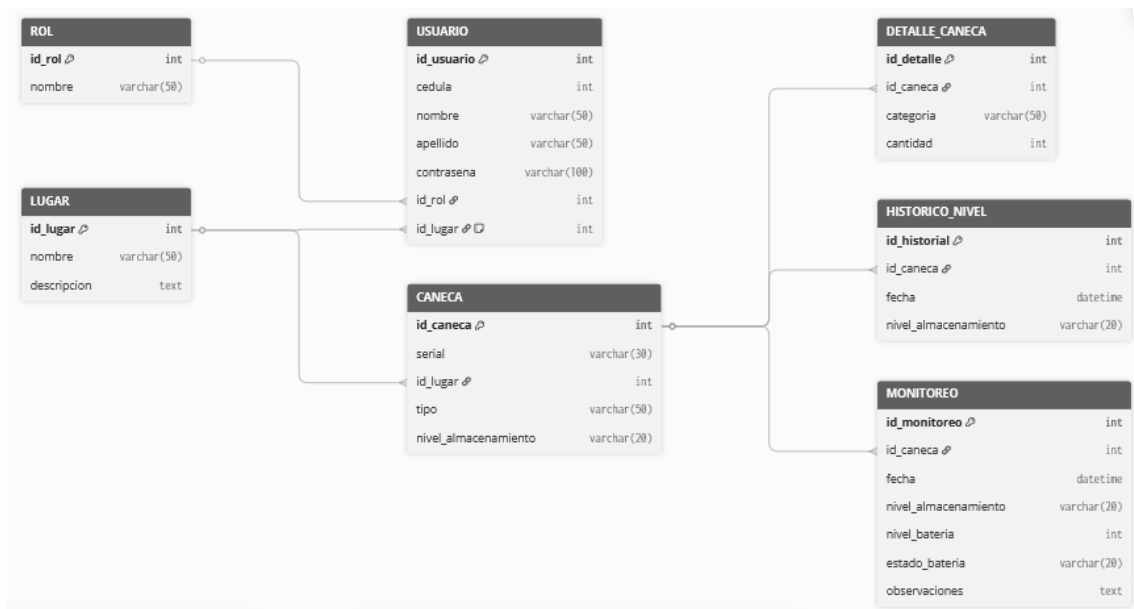
```
{
  "mensaje": "Caneca eliminada",
  "caneca": {
    "nombre": "Caneca vidrio",
    "botellas": 4,
    "otros": 1,
    "id": 4    }}
}
```


Lo cual las tablas quedarían así si las verificamos con **GET**:

Identificación: 1	Identificación: 2	Identificación: 3
Caneca: plásticos	Caneca: papel y cartón	Caneca: metal
Botellas: 10	Papeles: 10	Latas: 1
Bolsas: 3	Cartones: 2	Otros: 0
Otros: 5	Otros: 2	

- **Diagrama de base de datos:**

Hicimos uso de la herramienta **dbdiagram.io** para realizar un diagrama que indique nuestra secuencia de la base de datos, indicando que se debe iniciar registrando los roles, para que después el usuario al ingresar, señale uno de los dos roles a representar (administrador - visitante), también debe indicar desde que lugar se esta registrando, porque dependiendo del id del lugar, se hallaran ciertas canecas, y con ellas, sus características como el monitoreo de nivel de batería, de nivel de almacenamiento, el detalle de su contenido, etc. No debemos olvidar que las funciones del rol de administrador son diferentes a las del visitante, ya que, se podrán editar, añadir o eliminar las tablas dependiendo del rol del usuario, esto después de que el mismo haya ingresado con una contraseña.



- **Integración con MongoDB:**

Recordemos que MongoDB, es una base de datos NoSQL orientada a documentos, almacenamiento de datos en formato **JSON** o **BSON** (formatos que se usaron en los ejercicios con Postman), se almacenan los datos no en tablas, sino en colecciones con documentos, las cuales son útiles al momento de aplicarse en entornos web, IoT y microservicios. Para comenzar a usar esta herramienta, es necesario crear una cuenta en su página oficial: <https://www.mongodb.com> para así crear nuestro propio **cluster** (un grupo de servidores que permiten almacenar y gestionar datos).

Project Name	Clusters	Tags ⓘ	Users	Teams	Alerts	Actions
Project 0	1 Cluster	+ Add Tags	1 User	0 Teams	0 Alerts	⋮ 🗑

La propia plataforma nos indica los pasos a seguir, como por ejemplo, indicando el link de conexión desde el código del proyecto al cluster. En nuestro caso es:

`mongodb+srv://tequilaypachita_db_user:<db_password>@cluster0.bithaeo.mongodb.net/?appName=Cluster0`

Claramente tenemos que tener en cuenta la contraseña de nuestro usuario en MongoDB para la implementación correcta del sistema.

La implementación de esta base de datos a nuestro código requiere de una instalación previa desde el terminal. Para ello, crearemos un nuevo archivo para iniciar el código de nuestro programa, esto reuniendo los pasos de instalación de las anteriores actividades en clase (5 y 6). Cuando estemos instalando las dependencias necesarias, añadiremos la librería para la conexión con mongodb, esta se llama “**mongoose**” y se instala con la siguiente instrucción: “**npm install mongoose**”.

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\BackendDise2>npm install mongoose
added 17 packages, and audited 88 packages in 5s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

También implementaremos los servicios **JWT (JSON Web Token)**, los cuales, nos permiten manejar la autenticación y autorización en aplicaciones web, móviles o APIs, estos nos permiten generar “tokens” que son una serie alfanumérica que nos permitirá autorizar procesos, similar a una contraseña. Su paquete se denomina “**jsonwebtoken**” y su línea de instalación es la siguiente: “**npm install jsonwebtoken**”.

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\BackendDise2>npm install jsonwebtoken
added 13 packages, and audited 101 packages in 4s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

También instalaremos la librería de JavaScript “**bcryptjs**” que nos permitirá encriptar contraseñas de forma segura en aplicaciones web. Se usa mayormente para proteger los datos de acceso de los usuarios.

```
D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\BackendDise2>npm install bcryptjs
added 1 package, and audited 102 packages in 2s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

El código de ejemplo usado para verificar la conexión entre el programa en VisualStudio y el MongoDB es el siguiente.

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors')

const app = express();
//const port = 5000;

app.use(cors())
app.use(express.json())

const MONGO_URI = "mongodb+srv://tequilaypachita_db_user:<db_password>@cluster0.
bithaeo.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0"

mongoose.connect(MONGO_URI, {
  dbname: "cafeteria"
}).then(()=>{
  console.log("conectado a mongo atlas");
}).catch((err)=>{
  console.log("error conectado a mongo", err.message);
})

const usuarioSchema = new mongoose.Schema({
  cedula : {type: String, required:true},
  nombre : {type: String, required:true},
  apellido : {type: String, required:true},
  clave : {type: String, required:true},
  fecha : {type: Date, default: Date.now},
  rol : {type: String, required:true},
},
{
  timestamps : true
})

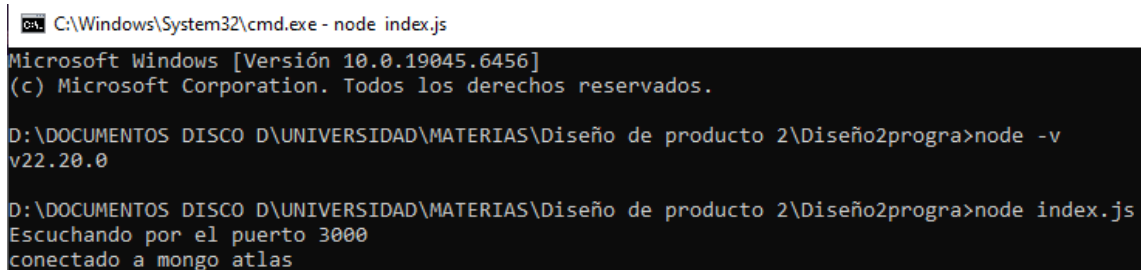
usuarioSchema.index({cedula: 1}, {unique: true})
```

```
const Usuario = mongoose.model('Usuario', usuarioSchema)

app.get('/', (req, res) => {
  var payload = {
    mensaje: "Hola Mundo!",
  }
  res.send(payload);
});

app.listen(3000, ()=>{
  console.log("Escuchando por el puerto 3000");
})
```

Si la implementación del código y el enlace al servidor con la base de datos, fueron satisfactorias se podría visualizar en el terminal con el mensaje “conectado a mongo atlas” de la siguiente manera.



```
C:\Windows\System32\cmd.exe - node index.js
Microsoft Windows [Versión 10.0.19045.6456]
(c) Microsoft Corporation. Todos los derechos reservados.

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\Diseño2progra>node -v
v22.20.0

D:\DOCUMENTOS DISCO D\UNIVERSIDAD\MATERIAS\Diseño de producto 2\Diseño2progra>node index.js
Escuchando por el puerto 3000
conectado a mongo atlas
```

Volviendo a la secuencia de base de datos que hicimos con dbdiagram.io, realizaremos un nuevo proyecto con un nuevo código que realice el funcionamiento mostrado en aquella gráfica, esto instalando las dependencias necesarias vistas anteriormente, pero con la diferencia de que incluiremos la conexión a mongodb. Usaremos las siguientes rutas para los diferentes servidores.

- /roles
- /lugares
- /usuarios
- /canecas
- /detallecanecas
- /monitoreos
- /historicos

Estos con las siguientes rutas principales (CRUD):

- Roles
 - POST /roles → Crear un rol
 - GET /roles → Ver roles
 - DELETE /roles/:id → Eliminar (solo admin. Requiere token)

- Lugares
 - POST /lugares → Crear un lugar
 - GET /lugares → Ver lugares
 - DELETE /lugares/:id → Eliminar (solo si no tiene caneca)
- Canecas
 - POST /canecas → Crear caneca
 - GET /canecas → Ver canecas
 - DELETE /canecas/:id → Eliminar (solo admin.)
- Monitoreo
 - POST /monitoreo → Crear monitoreo
 - GET /monitoreo → Ver monitoreo
 - GET /monitoreo/:id → Ver monitoreo de caneca específica
- Historico
 - POST /historico → Crear historico
 - GET /historico → Ver historico
 - GET /historico/:id → Ver historico de caneca específica

Un ejemplo de uso se explicará a continuación.

1. Primero se crea un rol con: POST <https://localhost:3000/roles> teniendo en cuenta la creación de un header de content-type application/json para que lo que se escriba en json quede registrado.

Headers 8 hidden

	Key	Value
☒	Content-Type	application/json
	Key	Value

Y escribimos en el Body – JSON la siguiente instrucción para crear el administrador.

```
{
  "nombre": "Administrador"
}
```

Esto para que el postman nos de un id

```
{
  "_id": "69048c5d8bd14b5e198d7261",
  "nombre": "Administrador",
  "__v": 0
}
```

Con esta información podemos proseguir, hay que guardar el número id.

2. Para crear un usuario necesitaremos del id creado para el rol, entraremos a POST <http://localhost:3000/usuarios> y en el JSON escribimos los datos a registrar.

```
{
  "cedula": "123456789",
  "nombre": "Angie",
  "apellido": "Martinez",
  "contrasena": "reciclaje",
  "rol": "69048c5d8bd14b5e198d7261"
}
```

Y nos debería de aparecer como respuesta lo siguiente.

```
{
  "mensaje": "Usuario creado",
  "usuario": {
    "cedula": "123456789",
    "nombre": "Angie",
    "apellido": "Martinez",
    "contrasena": "$2b$10$tmaYK3pn00DKkHn9f20.z0TroymtyPKN.4k.xPMMSwiLequ6MS7S",
    "rol": "69048c5d8bd14b5e198d7261",
    "_id": "69048dca8bd14b5e198d7263",
    "createdAt": "2025-10-31T10:22:02.381Z",
    "updatedAt": "2025-10-31T10:22:02.381Z",
    "__v": 0
  }
}
```

3. Para ingresar con el usuario creado entraremos a POST <http://localhost:3000/login> donde en el body ingresaremos nuestro usuario que es la cedula y nuestra contraseña.

```
{
  "cedula": "123456789",
  "contrasena": "reciclaje"
}
```

Y como resultado además de darnos el mensaje de ingreso exitoso, nos darán una serie alfanumerica para el token, esto dado por el JWT, este lo tendremos en cuenta para la autorización en acciones como la creación o borrado de elementos.

```
{
  "mensaje": "Login exitoso",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5MDQ4ZGNhOGJkMTRiNWUxOTlkNzI2MyIsInJvbiI6IkpkbWludXN0cmFkb3IiLCJpYXQiOiJlE3NjE5MDYyNDksImV4cCI6MTc2MTkxMzQ0OX0.QNMqIgcLb0y5OuVHsWtQ9Fq50AJxcyF-aB1MSkS1d24",
  "rol": "Administrador"
}
```

4. Con el token adquirido, procedemos a guardarlo como un header más, en el apartado de Authorization.

Headers 8 hidden

	Key	Value
☒	Content-Type	application/json
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
	Key	Value

Esto para seguir con la creación del lugar donde la caneca va a estar ubicada con POST <http://localhost:3000/lugares>, y escribimos sus datos en su Body.

```
{
  "nombre": "Bloque A",
  "descripcion": "Zona principal de recolección"
}
```

Para recibir como respuesta lo siguiente.

```
{
  "nombre": "Bloque A",
  "descripcion": "Zona principal de recolección",
  "_id": "69048ead8bd14b5e198d726c",
  "__v": 0
}
```

5. Procedemos en la creación de la caneca, como este es un ejemplo vamos a crear solo una con POST <http://localhost:3000/canecas> con el ingreso de sus datos en el JSON, tenemos que tener en cuenta el id obtenido anteriormente del lugar creado, para así registrar la caneca en dicho lugar.

```
{
  "serial": "PL-001",
  "id_lugar": "69048ead8bd14b5e198d726c",
  "tipo": "Plástico",
  "nivel_almacenamiento": "MEDIO"
}
```

Y su confirmación como resultado.

```
{
  "serial": "PL-001",
  "id_lugar": "69048ead8bd14b5e198d726c",
  "tipo": "Plástico",
  "nivel_almacenamiento": "MEDIO",
  "_id": "69048f098bd14b5e198d7270",
  "__v": 0
}
```

6. Ingresaremos más detalles de la caneca creada con POST <http://localhost:3000/detallecaneca> en su body.

```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "categoria": "botellas",
  "cantidad": 12
}
```

Con la confirmación de su creación como resultado.

```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "categoria": "botellas",
  "cantidad": 12,
  "_id": "69048f458bd14b5e198d7274",
  "__v": 0
}
```


7. También podemos revisar y editar el monitoreo, en este caso como no tenemos ningún registro, lo vamos a crear con POST <http://localhost:3000/monitoreo>.

```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "nivel_almacenamiento": "LLEN0",
  "nivel_bateria": 82,
  "estado_bateria": "NORMAL",
  "observaciones": "Sensor OK"
}
```

Y su confirmación.

```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "nivel_almacenamiento": "LLEN0",
  "nivel_bateria": 82,
  "estado_bateria": "NORMAL",
  "observaciones": "Sensor OK",
  "_id": "69048fc68bd14b5e198d727a",
  "fecha": "2025-10-31T10:30:30.016Z",
  "__v": 0
}
```

8. De igual manera, podemos hacer el mismo proceso del monitoreo con el historico con: POST <http://localhost:3000/historico>.

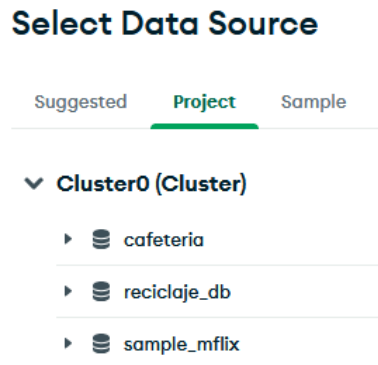
```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "nivel_almacenamiento": "LLEN0"
}
```

Y su confirmación a continuación.

```
{
  "id_caneca": "69048f098bd14b5e198d7270",
  "nivel_almacenamiento": "LLEN0",
  "_id": "69048ffe8bd14b5e198d727d",
  "fecha": "2025-10-31T10:31:26.362Z",
  "__v": 0
}
```

Si ingresamos más datos a la base de datos de mongodb, podemos visualizar gráficas y estadísticas que nos permitirán visualizar con mayor eficiencia los

resultados y el proceso del proyecto. Mediante la página de <https://charts.mongodb.com>.



Conclusiones:

El proyecto *Reciclajeficiente* se presenta como una solución tecnológica viable, escalable y con fuerte impacto social y ambiental. La integración de sensores, visión artificial y sistemas de clasificación automática permite un aporte significativo a la economía circular.

Gracias al análisis de requerimientos, la evaluación de mercado y el diseño arquitectónico, el proyecto tiene bases sólidas para su implementación y futuras mejoras.

Referencias:

- [1] C. Camacho, “Diseño Electrónico”, apuntes de clase, Diseño de Producto 2, Universidad Sergio Arboleda, Bogotá, Colombia, 2025.
- [2] Construye Fácil, “Cómo hacer una Cinta Transportadora (Planos PDF),” YouTube, 2019. [Online]. Available: <https://www.youtube.com/watch?v=PjL9drjZAQU>
- [3] Espressif Systems, “ESP32-WROOM-32 Datasheet,” 2020. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
- [4] R. Mischianti, “ESP32-wroom-32, esp32-S: flash, pinout, specs and IDE configuration – 1,” *Mischianti Electronics*, May 30, 2021. [Online]. Available: <https://mischianti.org/esp32-wroom-32-esp32-s-flash-pinout-specs-and-ide-configuration-1/>
- [5] “ESP32-WROOM-32 Pinout and Guide,” *Last Minute Engineers*, [Online]. Available: <https://lastminuteengineers.com/esp32-wroom-32-pinout-reference/>.
- [6] “Tutorial del controlador de motor paso a paso DRV8825 con Arduino,” *Last Minute Engineers*, [Online]. Available: <https://lastminuteengineers->

com.translate.google/dr8825-stepper-motor-driver-arduino-tutorial/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc.

- [7] “Arduino Nano ESP32 con controlador de motor paso a paso DRV8825,” *Newbiely*, [Online]. Available: <https://newbiely.com/tutorials/arduino-nano-esp32/arduino-nano-esp32-drv8825-stepper-motor-driver>.
- [8] “ESP32 and MAX7219 LED Matrix Display Tutorial,” *Makerguides*, [Online]. Available: <https://www.makerguides.com/esp32-and-max7219-matrix-display/>.
- [9] PCM – Protection Circuit Module,” *BMS PowerSafe*, 2016. [Online]. Available: <https://bmspowersafe.com/en/technology-en/pcm/>.
- [10] Argos, “Fuente/cargador con carga compartida (Load Sharing),” *Nómadas Electrónicos*, May 22, 2015. [Online]. Available: <https://nomadaselectronicos.wordpress.com/2015/05/22/fuentecargador-con-carga-compartida-load-sharing/>.
- [11] “DRV8825 Stepper Motor Driver Carrier, High Current,” *Pololu*, [Online]. Available: <https://www.pololu.com/product/2131>.
- [12] ElectroCrea, “Control de motor paso a paso con DRV8825 y ESP32,” *YouTube*, Jun. 17, 2023. [Online]. Available: <https://www.youtube.com/watch?v=PjL9drjZAQU>.
- [13] electrodeuna, “matrix-esp32,” *GitHub*, 2023 [Online]. Available: <https://github.com/electrodeuna/matrix-esp32>.
- [14] “Sensor de movimiento/proximidad por ultrasonido,” *ElectGPL Blogspot*, Dec. 22, 2010. [Online]. Available: <https://electgpl.blogspot.com/2010/12/sensor-de-movimiento-proximidad-por.html>.
- [15] ElectroCrea, “Control de matriz LED con MAX7219 y ESP32,” *YouTube*, Jul. 2, 2023. [Online]. Available: https://www.youtube.com/watch?v=PSFw_ipK3ug.
- [16] “Acceso a los Datos: Direcciones MAC e IP,” *CCNA desde Cero*, [Online]. Available: <https://ccnadesdecero.es/acceso-a-datos-direcciones/>